



**BOTSWANA INTERNATIONAL
UNIVERSITY
OF SCIENCE & TECHNOLOGY**

**EEEN520: Department of Electrical and
Communications Systems Engineering Project I
FINAL YEAR PROJECT I**

PROJECT TITLE:

Design a Machine Learning Based Advanced Predictive Framework for
Early Detection of Seizures for Epileptic Patients

NAME: THEBE B RATSATSI

STUDENT ID: 21000168

SUPERVISOR & Prof. Abid Yahya

Co-SUPERVISOR & DR MTENGI

PROGRAM: BEng ELECTRICAL& ELECTRONICS

DEPARTMENT: Department of Electrical and Communications Systems Engineering

DATE OF SUBMISSION:21/11/25

Contents

1		7
1.1	Introduction	7
1.2	Problem Statement	8
1.3	Aim	8
1.4	Objectives	9
2		10
2.1	LITERATURE REVIEW	10
2.1.1	Decision Trees	11
2.1.2	Supervised machine learning (ML) approaches to seizure prediction . .	12
2.1.3	Unsupervised seizure prediction methods	16
3		18
3.1	METHODOLOGY	18
3.1.1	Overview of the Methodology	18
3.1.2	System Modeling	18
3.1.3	How SVM helps Random Forest Limitations and Vice Versa	21
3.1.4	Data Acquisition	22
3.2	Signal Preprocessing	27
3.2.1	Resampling	28
3.2.2	Bandpass Filter	28
3.2.3	Normalizations Applied — Three Levels	28
3.3	Feature Extraction	29
3.3.1	Band Power (5 features per channel)	29
3.3.2	Hjorth Parameters (3 features per channel)	30
3.3.3	Statistical Features (5 features per channel)	30
3.4	Feature Selection — RF Importance (247 → 75)	30
3.5	Classification — Hybrid SVM + Random Forest	31
3.5.1	SVM with RBF Kernel	31
3.5.2	Random Forest	31
3.5.3	Weighted Ensemble Combination	32
3.5.4	Normal Class Threshold Correction	32
3.6	Final Prediction	33
3.7	System Implementation and Workflow	33
3.7.1	Workflow and Hardware	33
3.7.2	Hardware Components	34
3.8	Project Timeline	34
3.9	Project Budget	35

4		37
4.1	Implementation and Results	37
4.1.1	End-to-End System Flow	37
4.1.2	Proteus Simulation	37
4.1.3	LCD Display Behavior	38
4.1.4	Model Processing Pipeline behaviour	39
4.2	Table 3: LCD–Model Communication Link	40
4.2.1	Communication Flow	42
4.3	Hardware Pin Configuration	42
4.4	Monitoring System	49
4.4.1	Dashboard Overview	49
4.4.2	Sidebar and Global Controls	50
4.4.3	Ward Overview Tab	50
4.4.4	Patient Detail Tab	51
4.4.5	Ward Map Tab	53
4.4.6	Event Log Tab	55
4.4.7	Model Metrics Tab	56
4.5	Patient User-Interface(UI)	58
4.6	Simulation Results	59
4.6.1	Overview of System operation	59
4.7	Confusion Matrix and Performance Metrics	60
4.7.1	Confusion Matrix	60
4.8	Performance Metrics Explanation	61
4.9	Discussion	62
4.10	Recommendations	63
.1	Source Code	64
.1.1	LCD Controller	64
.1.2	EEG Detection Model	73

List of Figures

2.1	Decision Tree[?]	12
2.2	Machine learning Approach[?]	13
2.3	Illustration of hyperplane on SVM[?]	13
2.4	CNN-Bi-LSTM multi-channel fusion model based on attention mechanism.[?]	16
3.1	Workflow for EEG-based seizure detection using the hybrid SVM–RF model.	23
3.2	Electrode placements based on the international 10–20 system for EEG recordings.	24
3.3	End-to-end workflow and hardware architecture for hybrid SVM–RF seizure detection system.	35
3.4	Project timeline	36
4.1	System when OFF	42
4.2	System when LCD is powered On	43
4.3	LCD initializing	43
4.4	System waiting to Synchronize with model	44
4.5	System when Model is Syncing with LCD	44
4.6	System after Synchronization with LCD	45
4.7	System when it loads the dataset	45
4.8	system when it processes the Dataset	46
4.9	System when training	46
4.10	System after training and it outputs results, when there is No Seizure Normal	47
4.11	System before Seizure Occurs PRE SEIZURE	47
4.12	System when the Seizure Occurs SEIZURE	48
4.13	System after the seizure detection is done and it restarts RESTARTING	48
4.14	Simulation Circuit of the Seizure detection circuit	49
4.15	NeuroWatch dashboard home view showing the ward overview layout used for real-time clinical monitoring.	49
4.16	Ward Overview behaviour during a seizure event, showing the visual escalation used to draw clinician attention.	51
4.17	Patient Detail tab showing the per-patient diagnostic layout used for live monitoring and interpretation.	52
4.18	Feature-extraction and band-power visual components presented in the Patient Detail view.	53
4.19	Ward Map overview showing patient locations, hospitals, and distance context across Botswana.	54
4.20	Zoomed ward-map view for a selected patient with nearby facilities and exact location context.	54
4.21	Zoomed ward-map view for a selected patient with nearby facilities and exact location context.	55

4.22	Event-history visualisation used to inspect previous patient states and review state-transition patterns over time.	56
4.23	Dashboard model-metrics visualisation summarising the classifier performance measures available to clinicians and engineers.	57
4.24	Confusion matrix view used in the Model Metrics tab to inspect classification errors across the three EEG states.	57
4.25	Patient-facing user interface showing the smartwatch-inspired status view used to communicate EEG state and guidance in plain language.	58
4.26	Confusion Matrix of the Model	60
4.27	Performance Metrics of the Model	61

List of Tables

2.1	Comparison of EEG-Based Epilepsy Detection Methods[?]	12
2.2	Performance metrics at various train–test splits across decomposition levels [?].	14
2.3	Comparison of performance metrics for CHB-MIT and UCI datasets.	17
3.1	Comparison of limitations addressed by SVM, Random Forest, and the Hybrid Framework.	22
3.2	Class label definitions and segment counts in the Mendeley Epileptic EEG Dataset.	25
3.3	Window extraction scheme applied to each confirmed seizure event.	26
3.4	Comparison between the original distributed dataset and the reconstructed pre-ictal corpus.	27
3.5	List of Hardware Components Used in the System.	35
3.6	Estimated Budget for the project	35
4.1	Table 3.1: LCD Display Behavior (Raspberry Pi Code)	38
4.2	Table 3.2: ML Model Behavior (Python Detection Code)	39
4.3	LCD–Model Communication Interface	40
4.4	Hardware Pin Mapping	42

Chapter 1

1.1 Introduction

Epilepsy is a long-term neurological condition marked by recurring, unprovoked seizures, impacting around 70 million individuals globally [?]. These seizures arise from abrupt, irregular, and hyper-synchronous electrical activity in the cerebral cortex, which can be generally classified into various phases: the interictal (between seizures), pre-ictal (before a seizure), ictal (during a seizure), and post-ictal (after a seizure) periods [?]. The unpredictable nature of these seizures not only interferes with everyday activities but also brings significant risks of injury and premature death, highlighting the urgent need for effective early detection and prediction systems.

Electroencephalography (EEG) serves as a primary tool for investigating brain disorders, providing a direct, non-invasive measure of cortical activity with millisecond temporal resolution [?]. Visual examination of extended EEG recordings by qualified neurologists is the clinical gold standard for diagnosing epilepsy. However, because of the enormous volumes of data created, this approach is intrinsically subjective, expensive, time-consuming, and prone to human error [?]. A severe lack of neurological specialists makes this problem worse in settings with limited resources, delaying diagnosis and treatment. Decades of study on automated seizure detection and prediction utilising computer techniques have been motivated by these limitations. Early approaches relied heavily on signal processing techniques. While methods like Fourier Transform provided insights into characteristic frequency bands (delta, theta, alpha, beta), they were often inadequate for the non-stationary nature of EEG signals. The advent of time-frequency analysis tools, such as Wavelet Transforms, provided a more robust approach for detecting transient features characteristic of epileptiform activity, such as spikes and sharp waves [?]. Building on these advances, researchers began employing machine learning (ML) models—including neural networks [?] and decision trees [?—that relied on hand-crafted features extracted from EEG signals to perform classification tasks.

Today, the convergence of machine learning and edge artificial intelligence (AI) presents a transformative opportunity. Lightweight ML frameworks, including Graph Attention Networks, have demonstrated the potential for effective seizure detection on low-cost, accessible hardware, making them particularly suitable for resource-constrained environments. The primary goal of such systems is to accurately differentiate between pre-ictal and interictal states, enabling timely intervention. This project is situated within this evolving landscape. It aims to design and evaluate a lightweight, machine learning-based framework for the early detection of epileptic seizures using publicly available EEG datasets. The solution will be optimized for low-resource deployment and rigorously evaluated against established clinical benchmarks such as sensitivity, specificity, false alarm rate, and detection latency, contributing to the broader effort of making efficient and accessible neurological care a reality.

1.2 Problem Statement

Epilepsy remains a major public-health challenge in many low- and middle-income regions, including Botswana, where limited access to specialists, diagnostics, and continuous EEG monitoring results in delayed seizure detection, heightened risk of injury, and reduced quality of life. Traditional methods depend on specialized, expensive equipment and in-clinic supervision, making them unfeasible for widespread use in resource-constrained settings.

Recent studies underscore the potential of machine learning and edge AI to address these barriers. Lightweight frameworks, such as those proposed in [?], use Graph Attention Networks with accessible hardware to deliver effective detection performance in low-resource environments. This project addresses this critical need by developing and prototyping a machine-learning-based framework for early seizure detection using publicly available EEG datasets. The solution will emphasize low-resource deployment (e.g., on a laptop or edge device), and include transparent evaluation against metrics such as sensitivity, specificity, false alarm rate per hour, and detection latency, following benchmarks discussed in recent literature [?].

1.3 Aim

To design and evaluate a lightweight machine learning framework for early seizure detection using EEG signals, optimized for use in low-resource settings.

1.4 Objectives

1. To conduct a comprehensive review of existing seizure detection systems and predictive frameworks, particularly those utilizing EEG signals and machine learning.
2. To design and develop a machine learning-based predictive model tailored to early detection of seizures, using EEG data.
3. To implement a real-time or near real-time monitoring capability that enables continuous EEG surveillance and timely alerting for early seizure detection in low-resource and edge-deployment environments.
4. To evaluate the developed model's accuracy, reliability, and usability by testing it on relevant datasets, analyzing its performance metrics .

Chapter 2

2.1 LITERATURE REVIEW

With an increasing number of epileptic patients globally it is important to have improved epilepsy detection frameworks. The detection techniques for epileptic seizure using EEG vary depending on the specific characteristics of the dataset [?]. Timely detection through EEG is crucial yet challenging due to signal complexity, noise and manual analysis limitations. Some of the data detection methods of seizures were hand engineered techniques for feature extraction from the EEG data [?]. Therefore accurate feature extraction is crucial for improving the prediction and classification of epileptic seizures. Selecting of features that best describes EEG signal behaviour is important for automated seizure detection performance and different methods of seizure detection are used such as time domain[?], frequency domain[?], time frequency analysis[?], wavelength feature[?] and non linear signal analysis[?][?]. In a study that aims to differentiate normal seizures from epileptic seizures,[?] , the findings indicate that the majority of classification involve categorization or labeling of specific test instances. This process is achieved by employing a function that has learned to map distinct output variables or classes to their corresponding discrete input samples. A study by Mohammad Arlafaj[?] dwelled in investigating Deep Learning techniques used to improve epileptic seizure detection from EEG signals in wearable devices. Unlike prior studies relying on signal models or unbalanced datasets [?], this work compared long short term memory(LSTM variants, BiLSTN, stacked attention, convLSTM) alongside FCN and 2D CNN demonstrating SMOTE's superior impact on handling EEG's rarity of seizures. However data balancing techniques like SMOTE may introduce synthetic artifacts or discard valuable information[?] potentially degrading performance in noisy, real world ambulatory settings. Also high computational complexity of hybrid and attention based models such as ConvLSTM[?] poses challenges for deployment.

In a research that combines Deep learning(DL) and Machine Learning (ML) [?], EEG signals are preprocessed using a Butterworth Filter(0.5-50Hz) to remove noise, and decomposed into sub-bands via Discrete Wavelet Transform with an automated DL extraction feature, convolutional Neural Network. Relevant features are selected using mutual information estimators to reduce dimensionality. Classification was performed using machine learning algorithms such as Support vector machine(SVM)[?], K Nearest Neighbor (KNN), Linear Regression(LR), Random Forest(RF) and ANN. The model was later evaluated on the University of Bonn data set which is a single channel EEG dataset, with 500 short signal segments(23.6 second long) divided into 5 sets. Healthy people with eyes open(normal brain activity while alert), healthy people with eyes closed(in a relaxed state), inter-ictal signals in the opposite side of the brain(Non epileptic area of the brain), inter-ictal signals(epileptic area of the brain) and the ictal. It was also evaluated on the CHB-MIT dataset which is a multichannel with 1600 segments of 23 children with epilepsy with the purpose of distinguishing ictal and

pre-ictal to help in predicting oncoming seizures. This model performed exceptionally well in terms of accuracy, precision, sensitivity and F1 score(a balance between sensitivity and precision) however the small sample sizes dataset have limited instances e.g 500 for BONN potentially biasing results despite the cross validation and also the lack of exploration of decomposition methods, other techniques like Empirical Mode Decomposition(EMD) and Tunable Q Wavelet Transform(TQWT) might yield better results.

It is known that individuals with epilepsy also show hidden spikes in seizure free (resting state) EEG recordings compared to healthy people ,with unknown sub-bands[?][?]. Ricardo Buettner, Janek Frick and Thilo Rieg studied this and identified the EEG sub-bands(1.5-2Hz and 11-12.5Hz) as key differentiators in the seizure free recordings, enabling epilepsy detection without active seizures[?].However, their study relied on a limited dataset of 100 EEG recordings(50 from epileptic patients and 50 from healthy individuals) with unclear details on recording parameters,rather than using public datasets such as Bonn University or CHB-MIT.This approach risks bias due to the small dataset size and specific cohort(patients from a patients from a particular region, age group or clinical setting) , and lacks cross validation.

2.1.1 Decision Trees

Decision Trees classifiers are regarded to be a stand out most well known methods to data classification representation of classifiers. There are many different types of Decision tree algorithms, Classification and Regression Tree, Iterative Dichotomies 3 (ID3), Successor of ID3 (C4.5)[?], Multivariate Adaptive Regression Splines [?], Generalized, Unbiased, Interaction Detection and Estimation [?, ?] Classification Rule with Unbiased Interaction Selection and Estimation, Quick, Unbiased and Efficient Statistical Tree[?] .Their core principle involves recursively partitioning data based on feature values too create a tree like model for prediction .The strength of decision trees lies on their ability to handle non linear relationships and provide clear , human readable decision rules [?].A significant challenge with standard DTs is their tendency to overfit training data, leading to poor generalization, a limitation often addressed through techniques like pruning [?] .In a study that performed exceptionally well in terms of classification accuracy, the model achieved 99.12 for Binary, 98.89 for four class and 98.97 for eight class emotion classification on the DEAP dataset [?]. The ANT blends the interpretable, hierarchical decision-making of a tree structure with the feature learning capability of neural networks in a unique way. It works with raw or slightly processed EEG data, removing the requirement for manual feature extraction that is typical of older approaches, and its capacity to adapt to the data is a major improvement over static models. Reproducibility is difficult because of the complexity of the ANT architecture and the possibility that the implementation details required for replication are inadequate. Only the DEAP dataset is used to validate the model. It is uncertain how well it performs in more difficult, real-world situations with noise and inter-subject variability, or on different EEG emotion datasets. The remarkable accuracy may be unique to DEAP’s controlled environment.

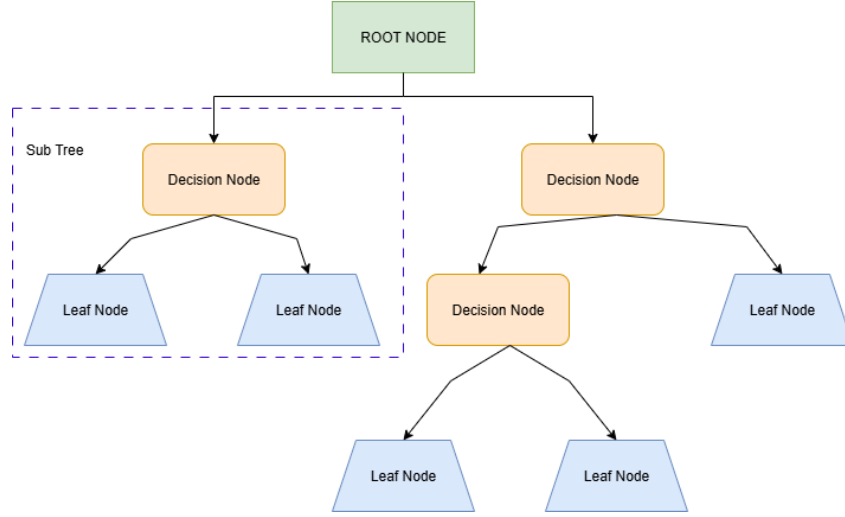


Figure 2.1: Decision Tree[?]

Kemel Polat and Gunes’s study focused on Hybrid Fast Fourier using Welch Method with C4.5 decision tree for binary detection[?]. With datasets from Bonn University , set A(healthy eyes open, 100 segments from 5 subjects) and set E (ictal hippocampus, 100 segments from 5 patients) each segment being 23.6 seconds. They observed a 98, accuracy (5 fold), 98.72 (10 fold), sensitivity of 98 specificity of 99 outperforming Recurrent Neural network(RNN) which is 96.75 and Adaptive Neuro-Fuzzy Inference System (ANFIS) 95. This performance emphasizes accuracy however it is tested on small data set of 200 segments biases toward the hippocampus seizures. There is no pre-processing which risks real world performance.

Table 2.1: Comparison of EEG-Based Epilepsy Detection Methods[?]

Author (year)	Method	Accuracy (%)
Guler[?] (2023)	Recurrent Neural Networks	96.79
Kannathal[?]	ANFIS Classifier	95
Subasi[?]	Wavelet-ME	95
Subasi[?]	Wavelet-MLPNN	93.6
Guler[?]	Wavelet-ANFIS	98.68
ourstudy	FFT-Decision Tree (10xFC)	98.72

2.1.2 Supervised machine learning (ML) approaches to seizure prediction

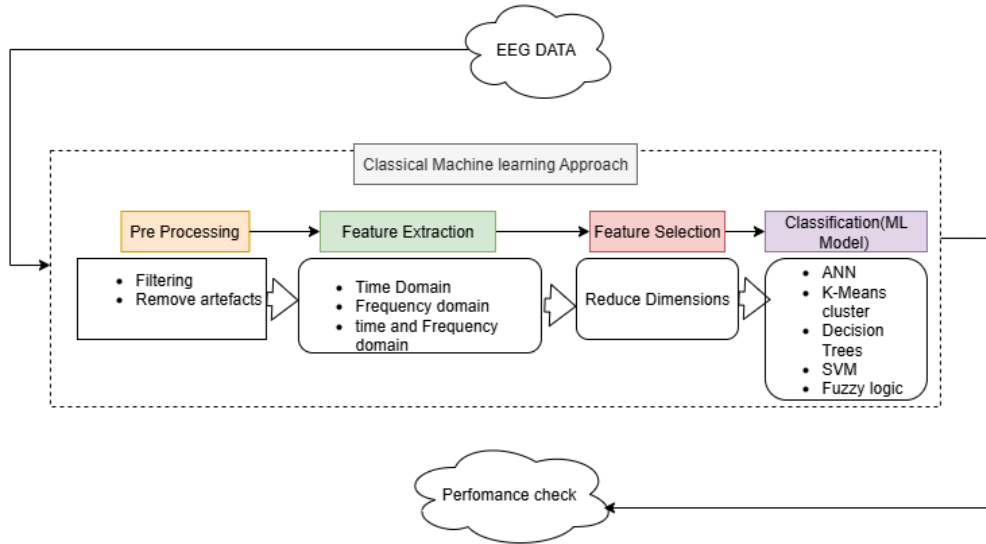


Figure 2.2: Machine learning Approach[?]

2.1.2.1 Support vector machines (SVMs)

The Support vector machine is a widely used classification method that was invented by Vapnik[?]. SVM is one of the techniques created to address classification and prediction issues that the traditional method is unable to resolve. The linear classifier principle was used in the development of SVM. SVM was created to address the nonlinear issue by introducing the kernel concept because the majority of cases do not match the linearity requirement. A study HSU by showed that classification using SVM will yield an accurate mapping[?]. Finding the ideal hyperplane on the input space is the concept of SVM. Two classes on the input space are separated by the hyperplane's function. Typically, -1 and +1 are used to indicate the classes. Figure 1 shows an example of the hyperplane on SVM. Rectangles display the pattern on class -1, whereas circles display the pattern on class +1. Separator lines (discriminant boundaries) between the two classes are depicted in Figure 2.3. With the largest margin hyperplane, the line is the best. The margin is the separation between each class's nearest pattern and the hyperplane. The support vector is the pattern that is closest. The circles in Figure 1b represent the support vector for every class. Furthermore, because the bold line is situated in the centre of the classes, it is the ideal hyperplane. SVM is based on the technique of determining the location of the ideal hyperplane.

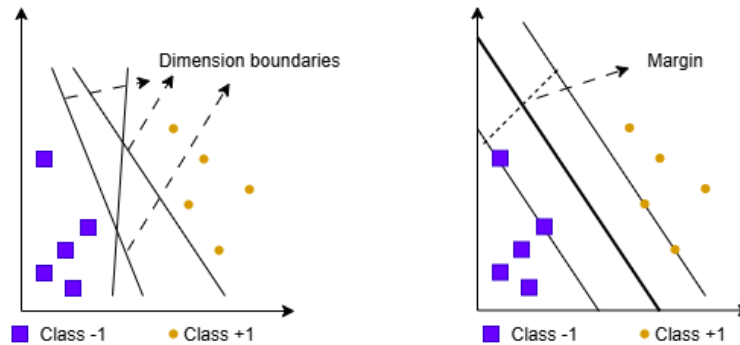


Figure 2.3: Illustration of hyperplane on SVM[?]

In order to overcome the inefficiency of manual EEG analysis, the SVM approach specifically uses Discrete Wavelet Transform (DWT) for signal pre-processing to convert EEG data into a format that is appropriate for classification. The SVM method breaks down raw EEG signals into frequency and temporal domains across levels (4, 6, and 8) by employing DWT with Daubechies order four (db4) wavelet. In order to prevent lossy feature extraction, this pre-processing phase creates wavelet coefficients (Details D1-Dl and Approximation A1) as predictors. To differentiate between epileptic and non-epileptic situations, the SVM classifier is applied to these coefficients using tuned parameters (e.g., $C=91$, $\gamma=0.302$ for 4-level decomposition). The study evaluates performance using a variety of train:test splits (75%: 25%, 80%: 20%, 85%: 15%, 90%: 10%, and 95% :5%), with the main parameter being classification accuracy (1-APER). Unlike RF, SVM does not rely on ensemble voting but optimizes a hyperplane to maximize the margin between classes, potentially benefiting from its robustness to high-dimensional data. The SVM achieved near perfect , with test accuracies ranging from 95.83% to 100% and training accuracies consistently above 98%. The highest test performance (100%) occur in imbalanced splits, 90%: 10%, 95% at level 8 decomposition, suggesting robustness with more training data. However , only accuracy is reported ,without sensitivity, specificity, or false positive rates ,which are essential for assessing clinical relevance, especially given epilepsy’s rare event nature. DWT captures frequency time information, but the study does not exploit temporal sequences, unlike the CNN-LSTM model with 94. 83% accuracy on 3-class CHB-MIT[?], highlighting a gap in dynamic signal modelling. The table below summarizes key SVM accuracy metrics from 4 level- 8.

Table 2.2: Performance metrics at various train–test splits across decomposition levels [?].

Train:Test Split	4-Level	6-Level	8-Level
75 : 25	99.73 / 99.19	98.37 / 97.54	99.73 / 99.19
80 : 20	99.75 / 96.90	98.73 / 97.96	99.75 / 98.99
85 : 15	99.53 / 100.00	98.81 / 95.83	99.76 / 100.00
90 : 10	99.55 / 97.96	98.65 / 95.83	99.77 / 100.00
95 : 5	99.36 / 100.00	98.50 / 100.00	99.79 / 100.00

2.1.2.2 Random forests

Random Forest (RF) is an ensemble learning method that constructs multiple decision trees during training and outputs the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees [?]. Through bootstrapping, random feature selection, and averaging, it excels at managing high-dimensional, noisy, and unbalanced data, which makes it especially useful for EEG-based seizure detection and prediction [?, ?, ?]. In contrast to deep learning models (like CNN-LSTM hybrids), RF offers great interpretability through feature importance scores (like Gini impurity reduction) that emphasise clinically relevant EEG bands (like delta variance during ictal states). It also doesn’t require a lot of preprocessing or GPU resources.

A study by Ziyu Hu[?] proposed a patient-specific RF model for seizure prediction using scalp EEG from the CHB-MIT dataset. The pipeline applied Discrete Wavelet Transform (DWT) to decompose signals into 7 sub-bands, extracted time-domain features (standard deviation, coefficient of variation [CV], log-amplitude, interquartile range), and used sliding windows

to expand pre-ictal samples. RF with CV from all sub-bands achieved 99.81% average accuracy across patients, with prediction horizons of 19.8–42.9 minutes. The work demonstrated that patient-specific models significantly outperform general models, and CV was the most discriminative feature for pre-ictal changes. Another study by Suvadeep Bose,[?] focused on three-class seizure detection (normal, interictal, ictal) using the Bonn dataset. DWT (Daubechies-4, level 4) with envelope analysis enhanced coefficient discriminability, followed by statistical feature extraction. RF outperformed MLP with 98% accuracy, validating envelope analysis as a novel pre-processing step to amplify seizure transients. Ashjan Basri and Muhammad Arif applied RF for multiclass seizure type classification (GNSZ, FNSZ, TCSZ vs. healthy) using FFT features from EEG montages [?] . Severe class imbalance was addressed via under-sampling (majority) and over-sampling (minority), yielding 96% accuracy demonstrating RF’s robustness in real-world clinical settings with uneven data distributions. However, several limitations remain unaddressed across these works. The reliance on controlled datasets—Bonn (small, single-channel) and CHB-MIT (pediatric, scalp-only)—limits generalizability to adult populations or real-world noisy environments with motion, hair, or EOG artifacts [?]. No study reported inference speed or power consumption, critical for real-time wearable deployment [?]. Moreover, RF was not benchmarked against hybrid models (e.g., RF-SVM or CNN-RF), which may offer superior non-linear boundary modeling or artifact robustness [?].

2.1.3 Unsupervised seizure prediction methods

2.1.3.1 Convolutional neural networks (CNNs)

Local features can be effectively extracted using the CNN-based approach. It performs well in feature extraction for noisy and non-stationary signals[?], but it has trouble identifying the time series data's global relevance. A recursive neural network variation called long short-term memory (LSTM) is frequently used to handle time series with nonlinear properties like EEG. It disregards local spatial information while taking into account the long-term interdependence of time series. In the majority of the literature, CNN and LSTM are regarded as end-to-end models [?, ?]. However, EEG signals in different routes show differently because not all parts of the brain produce aberrant discharges. Local characteristics are extracted by the CNN module, while time-series data features of the multi-channel EEG are captured by the Bi-LSTM module. The two modules are then combined in parallel and stored in a flatten layer. The attention-mechanism module is then used to choose the weight of the two-channel features after all the features have been combined into a completely connected layer[?].

A study proposed by Yahong MA and Zhentao[?], focused on EEG classification and prediction by integrating parallel CNN and Bi-LSTM branches for enhanced spatial and temporal feature extraction, surpassing single branch structures in accuracy through differentiated signal processing. By incorporating an attention mechanism post-fusion, the model optimally weights and filters features from important electrode channels, mitigating redundancy and boosting classification performance, as validated on CHB-MIT(94.83%) and UCI datasets(77.62%). Although the model performs well High accuracy on the CHB-MIT and moderate on UCI, possibly due to the preprocessed nature and five -class complexity raises questions about the generalizability across datasets or adult populations since also CHB-MIT is pediatric focused[?]. Also no details in runtime/inference speed, important for real time use. Attention weights could aid interpretability (identifying key channels) but are not visualized or analysed.

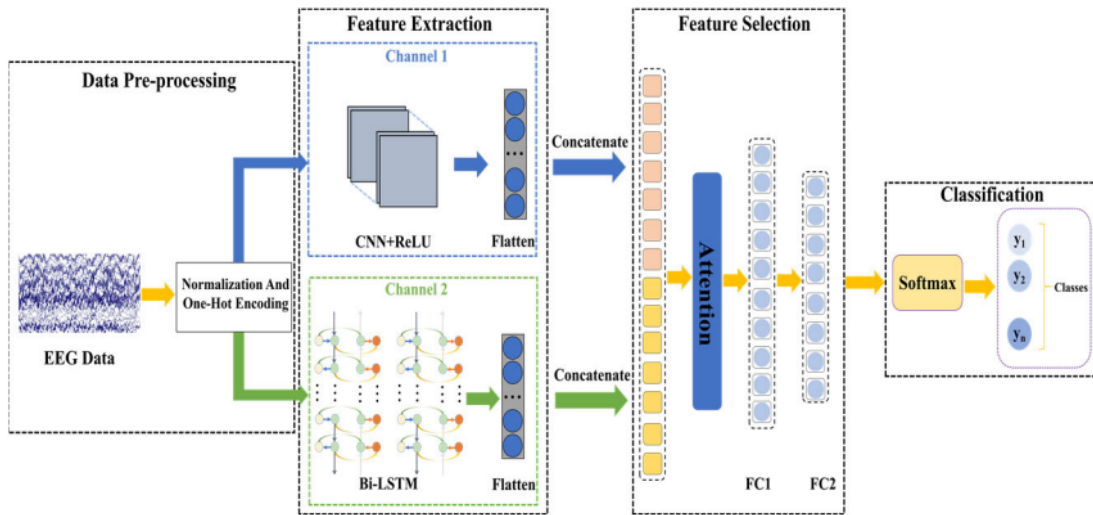


Figure 2.4: CNN-Bi-LSTM multi-channel fusion model based on attention mechanism.[?]

Table 2.3: Comparison of performance metrics for CHB-MIT and UCI datasets.

Metric	CHB-MIT (3 Classes)	UCI (5 Classes)
Accuracy (%)	94.83	77.62
Precision (%)	94.84	77.66
Recall (%)	94.84	77.62
F1-Score (%)	94.83	77.60
MCC (%)	92.26	72.03

Chapter 3

3.1 METHODOLOGY

3.1.1 Overview of the Methodology

This study adopts a structured methodology to design an advanced machine learning framework for seizure detection using EEG signals. The approach integrates two complementary algorithms—Random Forest (RF) and Support Vector Machine (SVM)—into a hybrid model that exploits the strengths of each while mitigating their individual shortcomings. By doing so, the proposed framework aims to enhance accuracy, robustness, and generalization, particularly in noisy and high-dimensional EEG data environments.

A wide range of classification algorithms has been applied in EEG-based seizure detection, including Linear Discriminant Analysis (LDA), Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Naïve Bayes, Logistic Regression, and Support Vector Machines (SVM). Among these, Random Forest has frequently demonstrated superior performance due to its ensemble nature and ability to handle noisy or imbalanced datasets [?]. Similar findings have been reported by Ikirigo and Suokiente [?], who highlighted RF’s reliability in biomedical classification tasks. However, RF alone suffers from issues such as bias toward majority classes and limited ability to capture complex, non-linear decision boundaries. Conversely, SVM excels in high-dimensional spaces and models intricate class separations, but its performance can degrade in the presence of noise and large datasets [?].

To address these complementary limitations, this project proposes a hybrid RF–SVM framework. In this design, Random Forest contributes robustness against noise, scalability, and feature selection capabilities, while SVM provides refined boundary optimization and enhanced discrimination in high-dimensional feature spaces. Together, this combination creates a balanced methodology capable of achieving higher predictive accuracy, reduced overfitting, and improved generalization compared to the use of either model in isolation.

3.1.2 System Modeling

1. Random Forest (RF)

Random Forest (RF), first introduced by Breiman [?], is an ensemble learning method that constructs multiple decision trees during training and aggregates their outputs to achieve higher predictive accuracy and stability. Each tree is trained on a different bootstrap sample of the dataset, and feature subsets are randomly selected at each node to ensure diversity among the trees. This ensemble approach reduces overfitting and enhances robustness, making RF particularly suitable for noisy and heterogeneous biomedical data such as EEG signals

[?].

RF produces either a majority vote (for classification tasks) or an average prediction (for regression tasks). Its ability to handle nonlinear relationships, coupled with its resilience to outliers, has established RF as a strong candidate for seizure detection frameworks [?]. Nevertheless, Breiman highlighted correlation-bound errors: when features are highly correlated, the performance of RF may deteriorate. Additionally, RF tends to favor majority classes, leading to bias in imbalanced datasets. To address this, Synthetic Minority Oversampling Techniques (SMOTE) and related balancing methods can be applied [?].

Algorithm Steps for a Single Tree

For each tree $t = 1, 2, \dots, T$ in the forest:

1. **Bagging:** Draw a bootstrap sample D_t (a random subset with replacement) from the original training data D .
2. **Random Feature Selection:** At each node, select a random subset of m features ($m < M$, where M is the total number of features).
3. **Node Splitting:** Split the node using the best criterion (e.g., Gini impurity or information gain) from the m selected features.
4. **Tree Growth:** Grow the tree to its maximum depth without pruning to maximize diversity among trees.

Aggregation of Predictions

For a new input $\mathbf{x}_{\text{new}}$, the Random Forest prediction $\hat{y}$ is obtained by majority vote of the individual tree predictions:

$$\hat{y} = \text{mode} \{T_1(\mathbf{x}_{\text{new}}), T_2(\mathbf{x}_{\text{new}}), \dots, T_T(\mathbf{x}_{\text{new}})\}, \quad (3.1)$$

where $T_t(\mathbf{x})$ represents the prediction of the t -th decision tree.

Key Limitations of Random Forest

Although Random Forest is powerful, it has notable limitations:

- When the number of features is very high, RF may struggle to isolate the most informative ones, particularly if many features are irrelevant or redundant.
- Decision boundaries are axis-aligned thresholds, which may not capture complex nonlinear separations as effectively as kernel-based methods.
- RF models can be biased toward majority classes in imbalanced datasets, necessitating the use of balancing techniques (e.g., SMOTE).

2. Support Vector Machine (SVM)

Support Vector Machine (SVM), originally developed by Vapnik [?], is a supervised learning algorithm widely used for classification and regression tasks. The fundamental idea behind SVM is to find an optimal hyperplane that separates data points from different classes with the maximum margin. This margin-based approach enhances generalization, particularly in high-dimensional feature spaces, making SVM highly effective for biomedical signal classification, including EEG data [?].

For linearly separable data, SVM identifies the hyperplane that maximizes the distance between the closest data points of each class, known as *support vectors*. For non-linearly separable problems, SVM employs the *kernel trick*, mapping the data into higher-dimensional feature spaces where linear separation becomes feasible. Common kernels include polynomial, sigmoid, and radial basis function (RBF), the latter being particularly effective for EEG signals with nonlinear patterns [?].

Mathematical Formulation of SVM

Decision Function & Constraints

For a given input feature vector $\mathbf{x}$, the decision function is expressed as:

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b, \quad (3.2)$$

where $\mathbf{w}$ is the weight vector and b is the bias term.

The classification rule is:

$$\text{Class}(\mathbf{x}) = \begin{cases} +1 & \text{if } f(\mathbf{x}) \geq 0, \\ -1 & \text{if } f(\mathbf{x}) < 0. \end{cases}$$

For a linearly separable dataset of N samples $\{(\mathbf{x}_i, d_i)\}$, where $d_i \in \{-1, +1\}$, the hyperplane must satisfy:

$$\mathbf{w}^T \mathbf{x}_i + b \geq +1 \quad \text{for } d_i = +1, \quad (3.3)$$

$$\mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad \text{for } d_i = -1. \quad (3.4)$$

These constraints can be combined into a single inequality:

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i. \quad (3.5)$$

Optimization Problem

The goal is to maximize the margin $\left(\frac{1}{\|\mathbf{w}\|}\right)$, formulated as a convex quadratic optimization problem:

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 \quad (3.6)$$

$$\text{subject to} \quad d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N. \quad (3.7)$$

This problem is typically solved using Lagrange multipliers, with the final solution depending only on the *support vectors*—the critical points lying closest to the decision boundary [?].

Key Limitations of Support Vector Machine

Despite its advantages, SVM has notable limitations:

- Training can be computationally expensive for very large datasets, as both memory usage and training time scale poorly with the number of samples.
- SVM is sensitive to outliers and noise, which may distort the optimal hyperplane and reduce generalization.
- Kernel and parameter selection (e.g., C , γ) require careful tuning, as inappropriate choices can lead to underfitting or overfitting.

3.1.3 How SVM helps Random Forest Limitations and Vice Versa

The integration of Random Forest (RF) and Support Vector Machine (SVM) into a hybrid framework is motivated by their complementary strengths. Each algorithm addresses specific shortcomings of the other, thereby improving overall model performance, robustness, and generalization. The benefits of this hybridization are outlined below.

Advantages Contributed by Random Forest:

- *Scalability*: RF is inherently parallelizable, making it suitable for large datasets. In the hybrid framework, RF can handle large-scale EEG data efficiently, while SVM is applied in later stages for refined boundary optimization [?].
- *Noise Robustness*: By averaging across multiple decision trees, RF naturally reduces the influence of noisy or outlier data points, thereby stabilizing the hybrid system's performance [?].
- *Feature Importance*: RF provides estimates of feature importance, which can be leveraged for dimensionality reduction. This reduces the computational burden on SVM, which is sensitive to large and high-dimensional input spaces.

Advantages Contributed by Support Vector Machine:

- *High-Dimensional Learning*: SVM is well-suited for high-dimensional spaces, making it capable of extracting meaningful patterns even when EEG features are numerous and complex [?, ?].
- *Complex Decision Boundaries*: With kernel functions such as the RBF kernel, SVM can capture nonlinear and intricate class boundaries that RF's axis-aligned splits may not fully represent [?].
- *Regularization and Overfitting Control*: The regularization parameter (C) enables SVM to balance margin maximization and classification error. When integrated with RF, this further reduces overfitting, especially in noisy EEG datasets [?].

RF contributes scalability, robustness, and feature selection, while SVM contributes precision, nonlinear modeling, and regularization. Together, they form a balanced framework where RF ensures computational efficiency and noise reduction, and SVM sharpens class separation for improved seizure detection accuracy.

Table 3.1: Comparison of limitations addressed by SVM, Random Forest, and the Hybrid Framework.

Limitation	SVM Ad- dresses	Random Forest Ad- dresses	Hybrid Out- come
High-dimensional feature spaces	Yes	No	Yes (SVM strength re- tained)
Scalability to large datasets	No	Yes	Yes (RF ensures efficiency)
Feature selection and rank- ing	No	Yes	Yes (RF reduces dimensionality before SVM)
Robustness to noisy/outlier data	No	Yes	Yes (noise dampened by RF)
Complex/nonlinear bound- aries	Yes	Limited	Yes (SVM ker- nels enhance decision bound- aries)
Model interpretability	Limited	Yes	Yes (RF im- proves inter- pretability; hybrid balances trade-off)

3.1.4 Data Acquisition

a) Workflow

The workflow for data acquisition and processing begins with EEG signal collection, followed by preprocessing, feature extraction, and classification using the proposed hybrid SVM–RF model. Figure 3.1 illustrates the overall pipeline of the methodology.

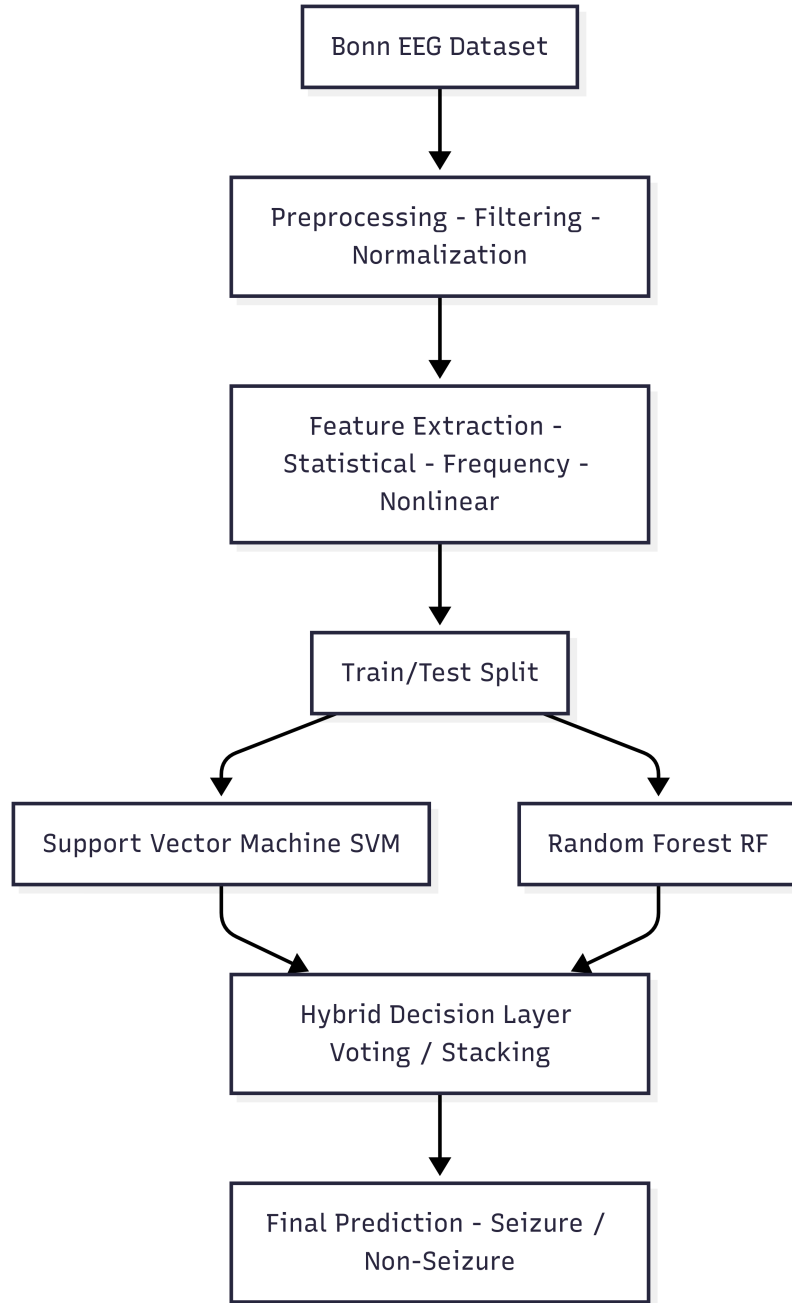


Figure 3.1: Workflow for EEG-based seizure detection using the hybrid SVM–RF model.

b) Mendeley Epileptic EEG Dataset

This study employs the publicly available Mendeley Epileptic EEG Dataset [?].

Dataset Description

The dataset contains the EEG recordings of six epileptic patients collected at the Epilepsy Monitoring Unit of the American University of Beirut Medical Center between January 2014 and July 2015. Recordings were made using 21 scalp electrodes following the 10-20 electrode placement system, sampled at 500 Hz. All channels were bandpass filtered between 0.625 Hz and 70 Hz with a 50 Hz notch filter applied to remove electrical interference. Some channels were omitted from specific recordings due to artefact constraints.

The data was obtained from patients diagnosed with focal epilepsy who were undergoing presurgical evaluation with long-term video-EEG monitoring. During this evaluation, anti-seizure medications were discontinued in order to record the patient’s habitual seizures. The dataset comprises 6 patients across a total of 20 recordings. Each recording includes an interictal portion during which no seizures occur and one or more ictal sections. Seizure type, ictal onset zone, and duration are labelled for each recording. The dataset contains 35 seizures in total, covering three seizure types: Complex Partial Seizures, Electrographic Seizures, and Video-Detected Seizures with no visual EEG change.



Raw Data Format

Labelled Data Format

24

segment is represented as a matrix of size 19×500 , where 19 is the number of channels and 500 samples corresponds to one second of recording at 500 Hz.

The class labels and their corresponding segment counts are given in Table 3.2.

Table 3.2: Class label definitions and segment counts in the Mendeley Epileptic EEG Dataset.

Label	Class	Segments	Duration (s)
0	Normal	3895	3895
1	Complex Partial Seizure	3034	3034
2	Electrographic Seizure	705	705
3	Video-Detected Seizure (no EEG visual change)	111	111
Total		7745	7745

The Normal class was sized to match the total duration of all seizure segments combined, creating a balanced distribution between normal and lesional data. The full labelled dataset has dimensions $7745 \times 19 \times 500$. This was split into a training set of 7011 samples (90%) and a test set of 779 samples (10%), saved in both NumPy (.`numpy`) and MATLAB (.`mat`) formats. The training set is stored as `x_train` of size $7011 \times 19 \times 500$ with its labels `y_train`, and the test set as `x_test` of size $779 \times 19 \times 500$ with its labels `y_test`.

Limitations of the Original Dataset Structure

The Mendeley Epileptic EEG Dataset is organised into four categorical labels: Normal, Complex Partial Seizure (CPS), Electrographic Seizure, and Video-Detected Seizure. While this structure is sufficient for binary seizure detection, it presents a fundamental problem for pre-ictal classification because no genuine pre-ictal class exists. The label described as “Pre-Seizure” in some works derived from this corpus is simply a relabelling of CPS activity, which is ongoing ictal or peri-ictal data, rather than the neurologically meaningful warning window that precedes seizure onset. The Video-Detected category captures events identified through clinical observation rather than measurable EEG change, which introduces noise into any EEG-based classifier. Most importantly, the pre-processed .`numpy` files distributed with the dataset discard the continuous temporal structure of the original recordings and replace it with fixed-length segments assigned to broad categories. This means that exact seizure timestamps, specifically the onset and offset times needed to isolate a genuine pre-ictal period, are lost entirely.

Reconstructing Pre-ictal Ground Truth from Raw EDF Files

To address these problems, a dedicated extraction pipeline was developed that bypasses the pre-processed files and operates directly on the raw EDF recordings. The pipeline uses a hand-curated seizure timestamp database (SEIZURE_DB), which maps each patient and recording to the exact onset time and duration of every confirmed seizure. For example, Patient 10, Recording 1 carries the annotation (07:36:38, 445 s), meaning a seizure begins at that time and lasts just under seven and a half minutes. These timestamps were verified against the accompanying clinical metadata and EDF channel annotations.

Each raw EDF file is loaded using MNE-Python, which preserves the recording’s absolute start time and allows seizure wall-clock times to be converted into sample-level indices. For every confirmed seizure, the pipeline extracts three non-overlapping window types, as summarised in Table 3.3.

Table 3.3: Window extraction scheme applied to each confirmed seizure event.

Class	Temporal Window	Label
Normal	More than 60 minutes from any seizure onset or offset	0
Pre-ictal	5 to 30 minutes before seizure onset	1
Ictal	Seizure onset to seizure offset	2

The pre-ictal window spans the interval from 30 minutes to 5 minutes before each seizure onset, giving a 25-minute segment that reflects the neurophysiological transition period before a seizure. The five-minute buffer between the window end and the onset marker ensures that no early ictal discharges contaminate the pre-ictal label.

Temporal Safety Constraints

Two safeguards are applied to prevent class contamination. First, when two seizures occur in close succession, the pre-ictal window of the later seizure is clipped so that it cannot overlap with the post-ictal recovery period of the earlier seizure. This prevents post-ictal suppression or early re-excitation from being mislabelled as pre-ictal activity. Second, Normal windows are subject to a strict 60-minute exclusion zone on both sides of every seizure. Any candidate Normal segment that falls within 60 minutes of either the onset or offset of any seizure in the recording is discarded. This ensures that interictal baseline windows are free from peri-ictal changes that could otherwise bias the classifier.

Class Balancing and Dataset Composition

Continuous recordings naturally produce far more interictal data than ictal or pre-ictal data, so a subsampling step is applied after extraction. All ictal windows are kept in full, and Normal and Pre-ictal windows are capped at a fixed multiple of the ictal count. This prevents the Normal class from dominating training and ensures the model sees a balanced representation of all three states. The final arrays are split into training and test sets and saved as `x_train.npy`, `y_train.npy`, `x_test.npy`, and `y_test.npy` under the `Npy_files_preictal/` directory.

Table 3.4 summarises the limitations of the original distributed dataset alongside the corresponding fixes applied in the reconstructed corpus.

Table 3.4: Comparison between the original distributed dataset and the reconstructed pre-ictal corpus.

Limitation of Original Dataset	Resolution in Reconstructed Corpus
No genuine pre-ictal class; the “Pre-Seizure” label reflects ongoing CPS activity	Pre-ictal windows are extracted strictly from the 5 to 30 minute period before confirmed onset
Video-Detected category includes events with no measurable EEG signature	Video-detected events are excluded; only EEG-confirmed seizure timestamps are used
Pre-processed .npy files discard continuous temporal structure	Raw EDF recordings are loaded directly, preserving absolute timestamps
Normal segments may contain peri-ictal neurophysiology	A 60-minute exclusion buffer on both sides of every seizure enforces clean Normal windows
No handling of closely spaced seizures	The pre-ictal window is clipped to the post-ictal boundary of any preceding seizure when necessary

3.2 Signal Preprocessing

Preprocessing of EEG signals is essential to enhance signal quality and remove artifacts. Three key steps are applied:

1. **Noise Removal:** Filtering is performed to remove artifacts such as muscle interference, powerline noise, and ocular movements.
2. **Normalization:** Standardization (Z-score normalization) ensures that the input features are scaled consistently across subjects and channels.
3. **Signal Preparation:** Segments are organized by frequency bands (Delta, Theta, Alpha, Beta, Gamma), which are physiologically linked to cognitive states and seizure activity.

The five primary EEG frequency bands are interpreted as follows:

- **Delta (0–4 Hz):** Dominant during deep sleep; abnormal presence in awake adults may indicate dysfunction or seizure focus.
- **Theta (4–8 Hz):** Associated with drowsiness, memory encoding, and pathological seizure states.
- **Alpha (8–13 Hz):** Resting rhythm observed in relaxed wakefulness; its disappearance or asymmetry may signal abnormalities.
- **Beta (13–30 Hz):** Linked to active concentration and motor activity; excessive beta may indicate anxiety or drug effects.
- **Gamma (30–45 Hz):** Related to sensory integration and cognitive processing; highly relevant in ictal (seizure) activity.

3.2.1 Resampling

The raw signals are resampled from the source sampling frequency to a target frequency:

$$f_s^{\text{target}} = 128 \text{ Hz}, \quad f_s^{\text{source}} = 500 \text{ Hz}.$$

Each channel signal is resampled by factor

$$r = \frac{f_s^{\text{target}}}{f_s^{\text{source}}}.$$

3.2.2 Bandpass Filter

EEG signals, being low-amplitude and highly susceptible to noise, require filtering prior to feature extraction. In this study, a Butterworth bandpass filter is employed, which offers a maximally flat frequency response in the passband. The magnitude-squared response of an analog Butterworth filter of order n is given by:

$$|H_n(j\omega)|^2 = \frac{1}{1 + \left(\frac{\omega^2 - \omega_0^2}{B\omega}\right)^{2n}},$$

where:

$$\begin{aligned} \omega &= 2\pi f \quad (\text{angular frequency}), \\ \omega_0 &= \sqrt{\omega_l \omega_h} \quad (\text{center frequency}), \\ B &= \omega_h - \omega_l \quad (\text{bandwidth}), \\ \omega_l &= 2\pi f_l \quad (\text{lower cutoff frequency}), \\ \omega_h &= 2\pi f_h \quad (\text{upper cutoff frequency}), \\ n &= \text{filter order}. \end{aligned}$$

A 4th-order Butterworth filter is applied forward and backward to achieve zero-phase filtering:

$$f_{\text{low}} = 0.5 \text{ Hz}, \quad f_{\text{high}} = 45.0 \text{ Hz}.$$

3.2.3 Normalizations Applied — Three Levels

3.2.3.1 Band Power Only

After computing raw band power P_b for each of the 5 bands, it is divided by the sum across all bands:

$$\hat{P}_b = \frac{P_b}{\sum_{b'=1}^5 P_{b'}}.$$

This makes band power scale-invariant — the relative distribution across bands matters, not the absolute amplitude.

3.2.3.2 StandardScaler — Full Feature Vector (Training Only)

After all 247 features are extracted, the scaler standardises each feature dimension across the training set:

$$z_j = \frac{f_j - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of feature j computed on the training split only. The same fitted scaler is applied to validation, test, and inference data. This is applied to SVM input only — the RF receives unscaled features.

3.2.3.3 Max-Abs Normalisation — Raw Signal at Inference Only

In `run_prediction.py` and `main_pi_bios_v15.py`, raw EDF amplitudes (microvolts) are normalised before feature extraction:

$$x_{\text{norm}} = \frac{x}{\max(|x|)}.$$

This normalisation is not applied during training because the `.npy` training files were already preprocessed to a consistent scale. It is applied at inference to align raw EDF input with the training distribution.

3.2.3.4 Complete Normalisation Chain

$$\begin{aligned} x_{\text{raw}} &\xrightarrow{\text{resample}} x_r \xrightarrow{\text{bandpass (BP only)}} x_{\text{flt}} \xrightarrow{\text{Welch}} S(f) \xrightarrow{\text{Option A}} \hat{P}_b \\ x_r &\xrightarrow{\text{Hjorth, Skew, Kurt, RMS, ZCR}} (\text{unfiltered stats}) \\ [\hat{P}_b, \text{Hjorth, stats, } H] &\in \mathbb{R}^{247} \xrightarrow{\text{StandardScaler}} \mathbf{z} \xrightarrow{\text{RF selector}} \mathbf{z}_{\text{sel}} \in \mathbb{R}^{75} \xrightarrow{\text{SVM}} \\ [\hat{P}_b, \text{Hjorth, stats, } H] &\in \mathbb{R}^{247} \xrightarrow{\text{RF selector}} \mathbf{f}_{\text{sel}} \in \mathbb{R}^{75} \xrightarrow{\text{RF}} \end{aligned}$$

3.3 Feature Extraction

From 19 channels, 13 features per channel are extracted, yielding a total of $19 \times 13 = 247$ features.

3.3.1 Band Power (5 features per channel)

The power spectral density (PSD) is estimated using Welch’s method with a segment length of 128 samples. Five physiological frequency bands are considered:

Band	Frequency Range (Hz)
Delta	1–4
Theta	4–8
Alpha	8–13
Beta	13–30
Gamma	30–45

The mean power in band b is

$$P_b = \frac{1}{|F_b|} \sum_{f \in F_b} S(f),$$

where $S(f)$ is the PSD and F_b is the set of frequencies in band b . Option A normalisation (relative band power, scale-invariant) is applied:

$$\hat{P}_b = \frac{P_b}{\sum_{b'} P_{b'}}.$$

3.3.2 Hjorth Parameters (3 features per channel)

For a signal x with first derivative x' and second derivative x'' :

$$\text{Activity} = \text{Var}(x), \quad (3.8)$$

$$\text{Mobility} = \sqrt{\frac{\text{Var}(x')}{\text{Var}(x)}}, \quad (3.9)$$

$$\text{Complexity} = \frac{\text{Mobility}(x')}{\text{Mobility}(x)}. \quad (3.10)$$

3.3.3 Statistical Features (5 features per channel)

- **Skewness:**

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(x - \mu)^3]}{(\mathbb{E}[(x - \mu)^2])^{3/2}}.$$

- **Excess Kurtosis:**

$$\gamma_2 = \frac{\mu_4}{\sigma^4} - 3 = \frac{\mathbb{E}[(x - \mu)^4]}{(\mathbb{E}[(x - \mu)^2])^2} - 3.$$

- **Root Mean Square:**

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{n=1}^N x_n^2}.$$

- **Zero-Crossing Rate:**

$$\text{ZCR} = \frac{1}{N-1} \sum_{n=1}^{N-1} \mathbf{1}[x_n \cdot x_{n+1} < 0].$$

- **Normalised Spectral Entropy:**

$$H = \frac{-\sum_f p(f) \log_2 p(f)}{\log_2 M}, \quad p(f) = \frac{S(f)}{\sum_{f'} S(f')},$$

where M is the number of frequency bins.

3.4 Feature Selection — RF Importance (247 → 75)

A bootstrap Random Forest with 300 trees and maximum depth 10 is trained on the full 247-feature set. The importance of feature j is the mean decrease in impurity across all trees and splits:

$$\hat{I}(j) = \frac{1}{|T|} \sum_{t \in T} \sum_{\substack{s \in t: \\ \text{split on } j}} p_s \cdot \Delta G_s,$$

where p_s is the fraction of samples reaching split s and ΔG_s is the Gini impurity decrease. The top 75 features by $\hat{I}(j)$ are retained.

3.5 Classification — Hybrid SVM + Random Forest

3.5.1 SVM with RBF Kernel

The decision function with class weights w_c solves the primal problem:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n w_{y_i} \xi_i$$

$$\text{subject to: } y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad \forall i,$$

where ξ_i are slack variables, $C = 1$, and class weights $w_c \in \{1.0, 1.8, 1.0\}$ for Normal, Pre-Seizure, Seizure respectively.

The equivalent dual problem (what the solver actually solves) is:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$

$$\text{subject to: } 0 \leq \alpha_i \leq w_{y_i} C, \quad \sum_{i=1}^n \alpha_i y_i = 0.$$

RBF kernel:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad \gamma = \frac{1}{d \cdot \text{Var}(\mathbf{X})},$$

where $d = 75$ (number of selected features) and $\text{Var}(\mathbf{X})$ is the variance of the scaled training data.

Decision function (multi-class via one-versus-one):

$$f_c(\mathbf{x}) = \sum_{i \in \text{SV}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b.$$

Probability calibration (Platt scaling) — converts raw scores to probabilities:

$$\hat{P}_{\text{SVM}}(c \mid \mathbf{x}) = \frac{1}{1 + \exp(A \cdot f_c(\mathbf{x}) + B)},$$

where A and B are fitted by maximum likelihood on a held-out fold during training.

3.5.2 Random Forest

An ensemble of $B = 500$ decision trees, each built on a bootstrap sample of the training data.

3.5.2.1 Bootstrap Sampling

For each tree b :

$$\mathcal{D}_b = \{(\mathbf{x}_i, y_i)\}_{i \in \mathcal{I}_b}, \quad \mathcal{I}_b \sim \text{Uniform}\{1, \dots, n\}^n \text{ with replacement.}$$

3.5.2.2 Node Splitting

At each node, a random subset of $m = \lfloor \sqrt{75} \rfloor = 8$ features is considered. The best split is chosen by minimising the weighted Gini impurity:

$$\Delta G = G(\mathcal{D}) - \frac{|\mathcal{D}_L|}{|\mathcal{D}|} G(\mathcal{D}_L) - \frac{|\mathcal{D}_R|}{|\mathcal{D}|} G(\mathcal{D}_R),$$

$$G(\mathcal{D}) = 1 - \sum_{c=0}^2 \hat{p}_c^2, \quad \hat{p}_c = \frac{\sum_{i \in \mathcal{D}} w_c \cdot \mathbf{1}[y_i = c]}{\sum_{i \in \mathcal{D}} w_{y_i}},$$

where w_c from `class_weight="balanced"`:

$$w_c = \frac{n}{K \cdot n_c}, \quad K = 3 \text{ classes}, \quad n_c = \text{count of class } c.$$

Stopping criteria: max depth = 5, min samples per leaf = 10.

3.5.2.3 Tree Probability Estimate

For class c at leaf ℓ :

$$\hat{P}_{T_b}(c | \mathbf{x}) = \frac{\sum_{i \in \ell} w_c \cdot \mathbf{1}[y_i = c]}{\sum_{i \in \ell} w_{y_i}}.$$

3.5.2.4 Forest Probability

Mean across all trees:

$$\hat{P}_{\text{RF}}(c | \mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{P}_{T_b}(c | \mathbf{x}).$$

3.5.3 Weighted Ensemble Combination

The outputs of both classifiers are combined as a weighted sum of their posterior probability estimates:

$$\hat{P}_{\text{ens}}(c | \mathbf{x}) = \alpha \cdot \hat{P}_{\text{SVM}}(c | \mathbf{x}) + (1 - \alpha) \cdot \hat{P}_{\text{RF}}(c | \mathbf{x}).$$

The blend weight α is selected by exhaustive search on the validation set:

$$\alpha^* = \arg \max_{\alpha \in \{0.05, 0.10, \dots, 0.95\}} \frac{1}{|\mathcal{D}_{\text{val}}|} \sum_{i \in \mathcal{D}_{\text{val}}} \mathbf{1} \left[\arg \max_c \hat{P}_{\text{ens}}(c | \mathbf{x}_i) = y_i \right].$$

Note on feature scaling: The two classifiers receive different inputs to the same selected feature set $\mathbf{f}_{\text{sel}} \in \mathbb{R}^{75}$:

SVM input: $\mathbf{z}_{\text{sel}} = \text{StandardScaler}(\mathbf{f}_{\text{sel}})$, RF input: $\mathbf{f}_{\text{sel}}$ (unscaled — RF is scale-invariant).

3.5.4 Normal Class Threshold Correction

Let $\hat{c} = \arg \max_c \hat{P}_{\text{ens}}(c | \mathbf{x})$ be the class with highest blended probability. A threshold τ is applied to reduce false alarms from Normal to Pre-Seizure without suppressing true Seizure detections:

$$\hat{y} = \begin{cases} \text{Normal} & \text{if } \hat{P}_{\text{ens}}(\text{Normal} | \mathbf{x}) \geq \tau \text{ and } \hat{c} = \text{Pre-Seizure} \\ \hat{c} & \text{otherwise.} \end{cases}$$

The threshold τ is selected by grid search over $[0.30, 0.65]$ (step 0.01) on the validation set, maximising overall accuracy.

3.6 Final Prediction

Given the blended probability vector $\hat{P}_{\text{ens}}(c \mid \mathbf{x})$ over classes $c \in \{0, 1, 2\}$ (Normal, Pre-Seizure, Seizure), the base prediction is:

$$\hat{y}_{\text{base}} = \arg \max_c \hat{P}_{\text{ens}}(c \mid \mathbf{x}).$$

The Normal threshold correction is then applied:

$$\hat{y} = \begin{cases} 0 \text{ (Normal)} & \text{if } \hat{y}_{\text{base}} = 1 \text{ (Pre-Seizure) and } \hat{P}_{\text{ens}}(0 \mid \mathbf{x}) \geq \tau^* \\ \hat{y}_{\text{base}} & \text{otherwise,} \end{cases}$$

where

$$\tau^* = \arg \max_{\tau \in [0.30, 0.65]} \text{Accuracy}(\mathcal{D}_{\text{val}}, \tau).$$

The reported confidence score is the probability of the final predicted class:

$$\text{confidence} = \hat{P}_{\text{ens}}(\hat{y} \mid \mathbf{x}).$$

The constraint that only Pre-Seizure predictions are eligible for override ensures Seizure detections are never suppressed — a sample classified as Seizure with any confidence level remains Seizure regardless of τ .

The complete decision pipeline is therefore:

$$\mathbf{x} \xrightarrow{\text{preprocess}} \mathbf{f} \in \mathbb{R}^{247} \xrightarrow{\text{RF importance}} \mathbf{f}_{\text{sel}} \in \mathbb{R}^{75} \xrightarrow{\text{SVM + RF blend}} \hat{P}_{\text{ens}}(c \mid \mathbf{x}) \xrightarrow{\arg \max} \hat{y}_{\text{base}} \xrightarrow{\tau\text{-correction}} \hat{y}.$$

3.7 System Implementation and Workflow

3.7.1 Workflow and Hardware

The implementation of the proposed seizure detection system operates as a remote monitoring pipeline, where EEG data is processed at an edge device and results are transmitted to a clinical dashboard accessible by the supervising physician. The workflow is designed for modularity, supporting both offline training and live file-based inference on a Raspberry Pi or Windows processing unit.

3.7.1.1 Hardware Components

- **EEG Acquisition:** A 19-channel EEG amplifier placed according to the international 10-20 system captures brain electrical activity at 500 Hz. For training and evaluation, the Mendeley Epileptic EEG Dataset was used — comprising EDF recordings from 6 patients with focal epilepsy at 500 Hz, containing 35 seizures across 19 channels.
- **Processing Unit:** A Raspberry Pi runs the inference engine (`main_pi_bios_v15.py`), performing real-time windowing, feature extraction, and classification. A laptop is used during training.
- **Output Interface:** A 20×4 character LCD displays the current patient state, confidence score, and elapsed time. GPIO-connected LEDs provide visual alerts (green = Normal, red = Pre-Seizure or Seizure), and a buzzer produces state-specific alarm patterns — a single pulse per second for Pre-Seizure and rapid continuous beeping for Seizure.

- **Remote Monitoring Interface:** The system writes a structured JSON status file after each prediction window. A clinical dashboard (`dashboard_v2.py`) running on the physician’s machine reads this file to display patient state, confidence trends, and GPS location — enabling monitoring of patients in remote areas without requiring physical bedside presence.

3.7.1.2 Workflow

The signal processing and classification pipeline proceeds through the following stages:

1. EEG signal acquisition from 19-channel EDF recordings at 500 Hz.
2. Non-overlapping 1-second windows extracted per channel, yielding windows of shape (19 channels $\times$ 500 samples).
3. Max-abs normalisation of the raw window to $[-1, 1]$ to align inference input with training data scale:

$$x_{\text{norm}} = \frac{x}{\max(|x|)}.$$

4. Per-channel resampling from 500 Hz to 128 Hz, followed by 4th-order zero-phase Butterworth bandpass filtering (0.5–45 Hz).
5. Feature extraction producing a 247-dimensional vector per window: 5 normalised band powers + 8 statistical and nonlinear features across all 19 channels.
6. RF importance-based feature selection reducing the vector from 247 to 75 features using a pre-fitted bootstrap Random Forest importance ranking.
7. Hybrid classification: SVM (RBF kernel, $C = 1$, trained on StandardScaler-normalised selected features) and Random Forest (500 trees, max depth 5, trained on unscaled selected features) produce independent class probability vectors, blended as

$$\hat{P}_{\text{ens}}(c \mid \mathbf{x}) = \alpha \hat{P}_{\text{SVM}}(c \mid \mathbf{x}) + (1 - \alpha) \hat{P}_{\text{RF}}(c \mid \mathbf{x}).$$

8. Normal threshold correction: Pre-Seizure predictions are downgraded to Normal if $\hat{P}_{\text{ens}}(\text{Normal} \mid \mathbf{x}) \geq \tau^*$; Seizure predictions are never overridden.
9. Decision output triggers the appropriate LCD message, LED state, buzzer pattern, JSON status update, and optional SMS alert to the patient’s emergency contact.

This architecture ensures efficient EEG signal acquisition and robust classification. Random Forest reduces dimensionality and enhances robustness to noise, while SVM refines decision boundaries, yielding accurate seizure detection. The modular design allows future integration with wearable devices for continuous monitoring in low-resource clinical settings.

3.7.2 Hardware Components

3.8 Project Timeline

The project timeline entails detailed plan that outlines the sequence of activities, milestones, and deadlines required to complete a project within a specific timeframe. The project timeline has been visually represented using a Gantt chart and it also shows the Risk Management Plan which is crucial to cater for delays.

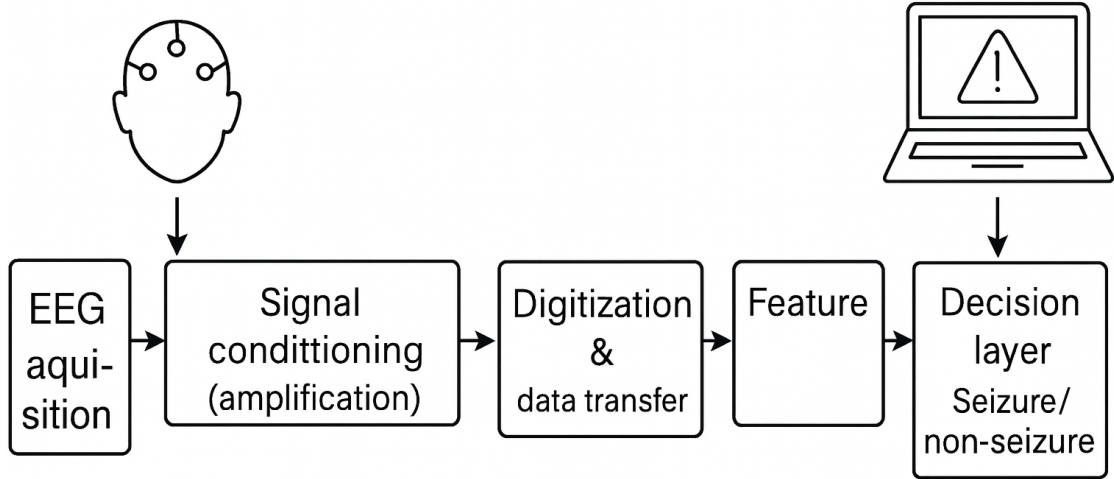


Figure 3.3: End-to-end workflow and hardware architecture for hybrid SVM–RF seizure detection system.

Table 3.5: List of Hardware Components Used in the System.

Component	Description / Function
Raspberry Pi 4	Serves as the central processing unit, executing the machine learning algorithm and managing all sensor and output interfaces.
LCD	Displays real-time system status, alerts, and results from the predictive framework.
LED	Provides visual feedback indicating system states such as normal operation, warning, or error conditions.
Buzzer	Emits audible signals to alert users during critical events or detected anomalies.

3.9 Project Budget

This section shows the estimated budget for all the expenses in the project. As shown in Table 6, the components to be used in the project are estimated to cost at about P1500 and P500 will be used to cover other unexpected expenses during the project.

Table 3.6: Estimated Budget for the project

Expenses	Estimated Cost(P)
Components	1500
Unexpected expenses	500
Total	2000

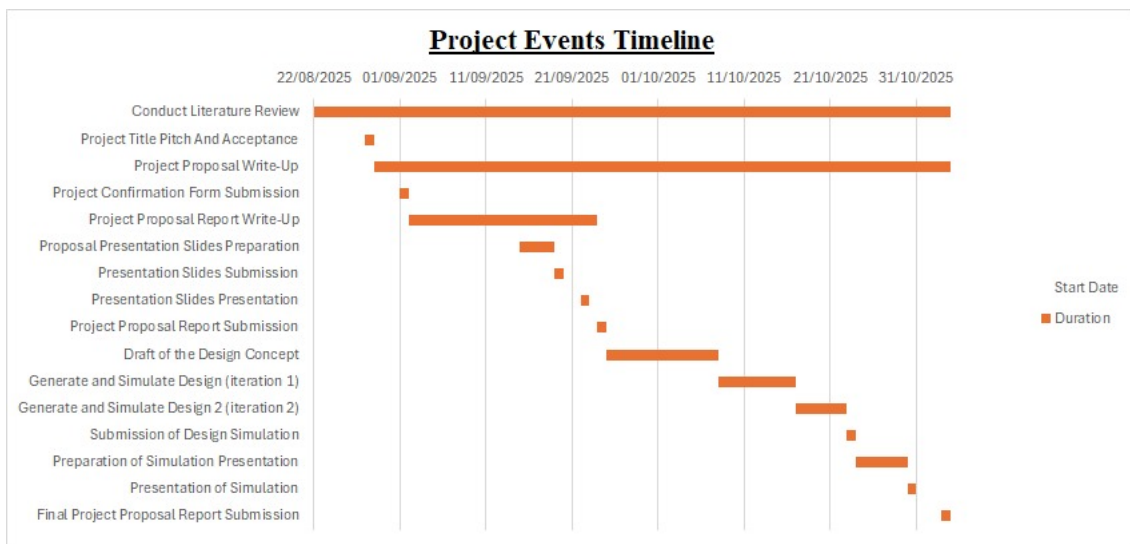
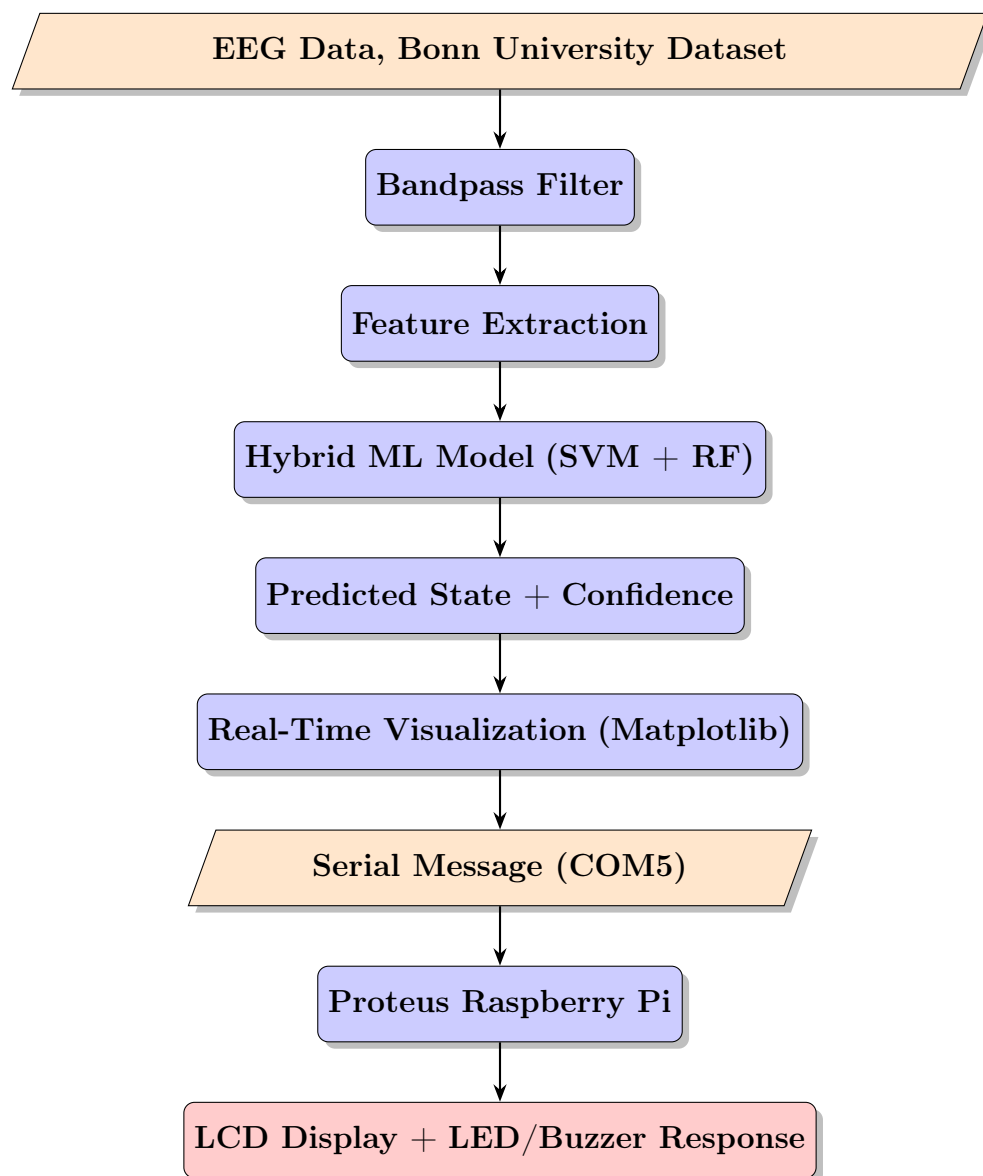


Figure 3.4: Project timeline

Chapter 4

4.1 Implementation and Results

4.1.1 End-to-End System Flow



4.1.2 Proteus Simulation

4.1.3 LCD Display Behavior

Table 4.1: Table 3.1: LCD Display Behavior (Raspberry Pi Code)

Hardware	Interactive	System	Boost	Display Timeline	OFF (LED)	OFF (Buzzer)	Initial Learning
Boot	1 s	LOADING...	(empty)	Blink	OFF	OFF	Initial Learning
Initializing	1.5 s	INITIALIZING	(empty)	Blink	OFF	OFF	Red indicates load
Loading	2 s	LOADING...	All Stable	Blink (occasional)	OFF	OFF	Flashing + RSD
Processing	2 s	PROCESSING...	Analyzing	Blink (occasional)	OFF	OFF	Pre-Seizure Analysis
Training	2 s	TRAINING...	Learning Model	Blink (occasional)	OFF	OFF	Training loop
Ready	1 s	SYSTEM READY	Starting Scan	Blink (occasional)	OFF	OFF	4-min cycle start
Normal	2 s	STATUS: NORMAL	All Stable	Blink (occasional)	OFF	OFF	Monitoring
Pre-Seizure	2 s	!PRE-SEIZURE!	Take Meds!	Blink (occasional)	Blink (yellow)	OFF	Pre-seizure buzzer option
Seizure	4+ min	!! SEIZURE !!	SEEK HELP!	Blink (continuous)	Blink (red)	Buzzer ON	4-min cycle
Restarting	1 s	RESTARTING	Please Wait	Blink	OFF	OFF	Resetting loop
Cycle Between	...	...	Returns to load	...	...	...	Loop

4.1.4 Model Processing Pipeline behaviour

Table 4.2: Table 3.2: ML Model Behavior (Python Detection Code)

Hardware	System Start	Processing Pipeline	Model—Notify	3 sec Delay	Notes
Initialization	...	Model—Notify: INITIALIZING	...	Rate: 5 orders	...
Initializing Data Loading	Read EEG Files, Extract	Data account... LOADING	...	Identify hardware...	...
Feature Processing	Extract + PSD	Bandpass Features Ready	PROCESSING	Processing	...
Initializing Training Model	Train SVM+RF (80–20 split)	Models Trained	TRAINING	One of 12 classes	...
Prediction	Classify seizure / non-seizure	Seizure — Syn.4+.45 Node Output	...	70	Predicted Output
Notification Detection (Normal)	pred = 0	...	STATUS: NORMAL	Green region	...
Pre-Seizure Detection	pred = 1	...	PRE-SEIZURE! WARNING	Yellow region	...
Seizure Detection	pred = 2	...	SEIZURE!! CRITICAL STATE	Red region (tested)	...
Final Metrics	Determine Model (all predictions)	Confusion Metrics (F*) Matrix,	FINAL	Final metrics + state cycle*	...
Map Compute (Final Cycle)	Map Area (Pre-predicted)	...	CYCLE_COMPLETE PHASE	Metrics + cycle summary	If enough samples

4.2 Table 3: LCD–Model Communication Link

Table 4.3: LCD–Model Communication Interface

Model Event	Serial Message	LCD Response	Sync	LED/Buzzer	Visual Feedback
System Boot	BOOT	"SYSTEM BOOTING Please Wait..."	Async	Green continuous blink	Model initializes
Initialization	INIT	"INITIALIZING Set- ting Up..."	Async	Green continuous blink	System setup
Data Loading	LOAD	"LOADING... Read- ing Data..." (animated dots)	Real-time	Green continuous blink	Files being read
Feature Extraction	PROC	"PROCESSING... An- alyzing..." (animated dots)	Per-sample	Green continuous blink	Feature extraction
Model Training	TRAIN	"TRAINING... Learn- ing Model" (animated dots)	Background	Green continuous blink	SVM/RF training
System Ready	RDY	"SYSTEM READY Starting Scan"	Aligned	Green continuous blink	Models trained
Normal Monitoring	NORM	"STATUS: NORMAL All Stable"	Real-time	Green continuous blink	Stable operation
Pre-Seizure Alert	WARN	"!PRE-SEIZURE! Take Meds Now!"	Real-time	Green blink + Red blink + Buzzer blink	Warning state

Model Event	Serial Message	LCD Response	Sync	LED/Buzzer	Visual Feedback
Seizure Detection	SEIZ	"!! SEIZURE !!" (blinking) + scrolling "SEEK MEDICAL HELP NOW!"	Real-time	Green blink Red blink Buzzer blink	Critical alarm
System Restart	REST	"RESTARTING Please Wait..."	Background	Green continuous blink	Cycle restart
Heartbeat Signal	HB	No display change	Background	Green continuous blink	Connection alive

4.2.1 Communication Flow

Model (Python) → Serial COM5 @ 9600 baud → Proteus Simulation → LCD Display

4.3 Hardware Pin Configuration

Table 4.4: Hardware Pin Mapping

Component	GPIO Pin (BCM)	Purpose
LCD RS	26	Register Select
LCD E	19	Enable
LCD D4-D7	13, 6, 5, 20	Data lines
Green LED	27	System heartbeat
Red LED	22	Warning indicator
Buzzer	4	Alarm output

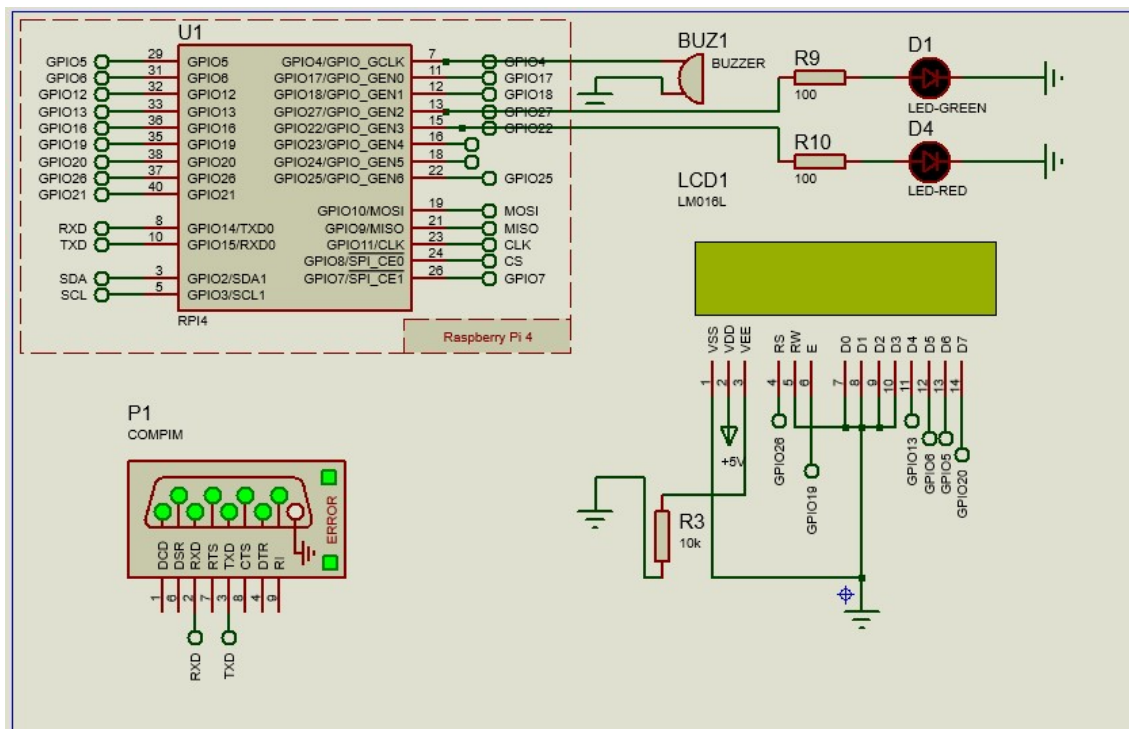


Figure 4.1: System when OFF

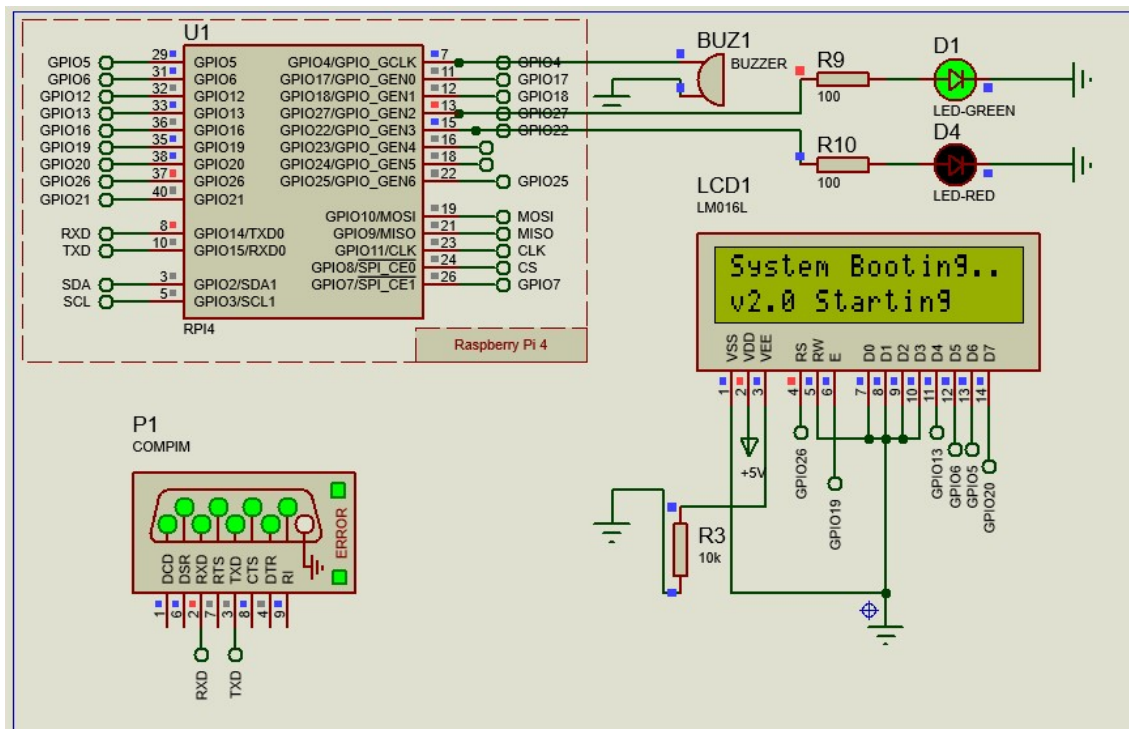


Figure 4.2: System when LCD is powered On

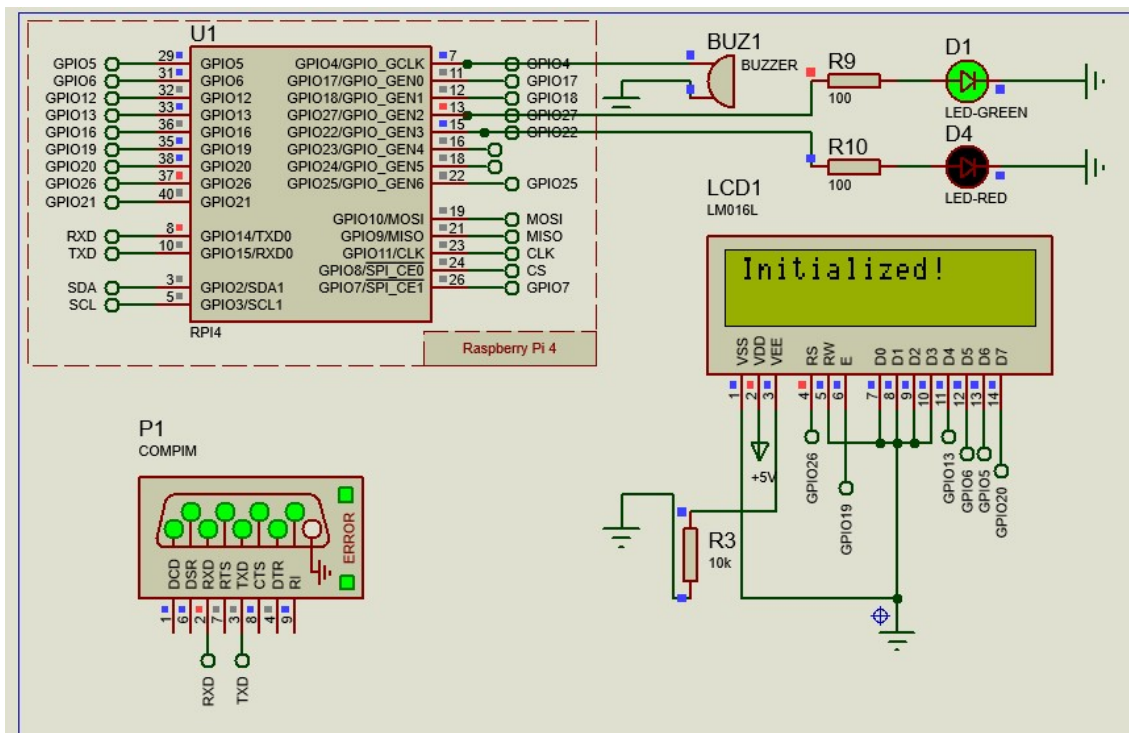


Figure 4.3: LCD initializing

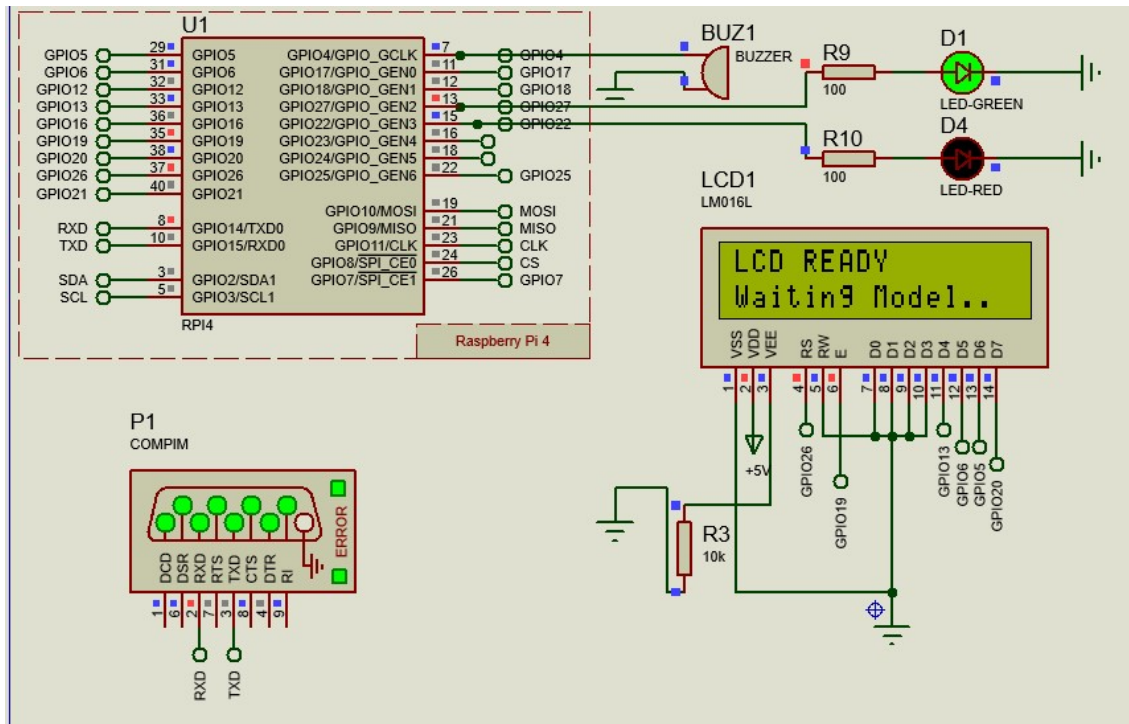


Figure 4.4: System waiting to Synchronize with model

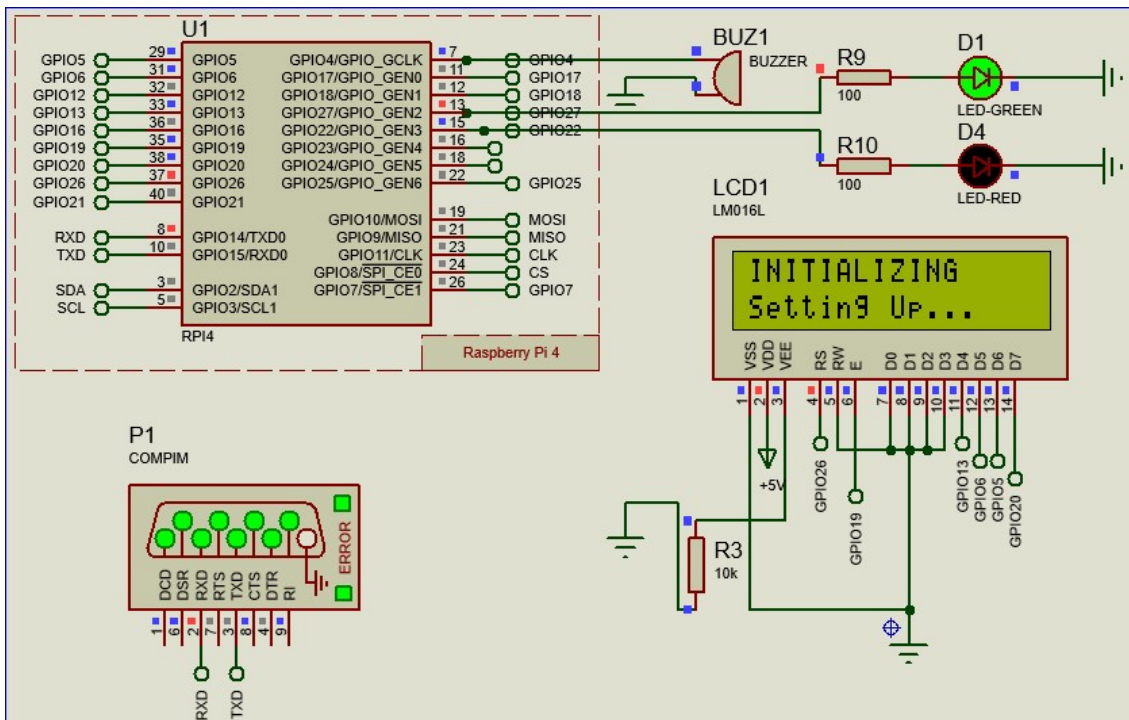


Figure 4.5: System when Model is Syncing with LCD

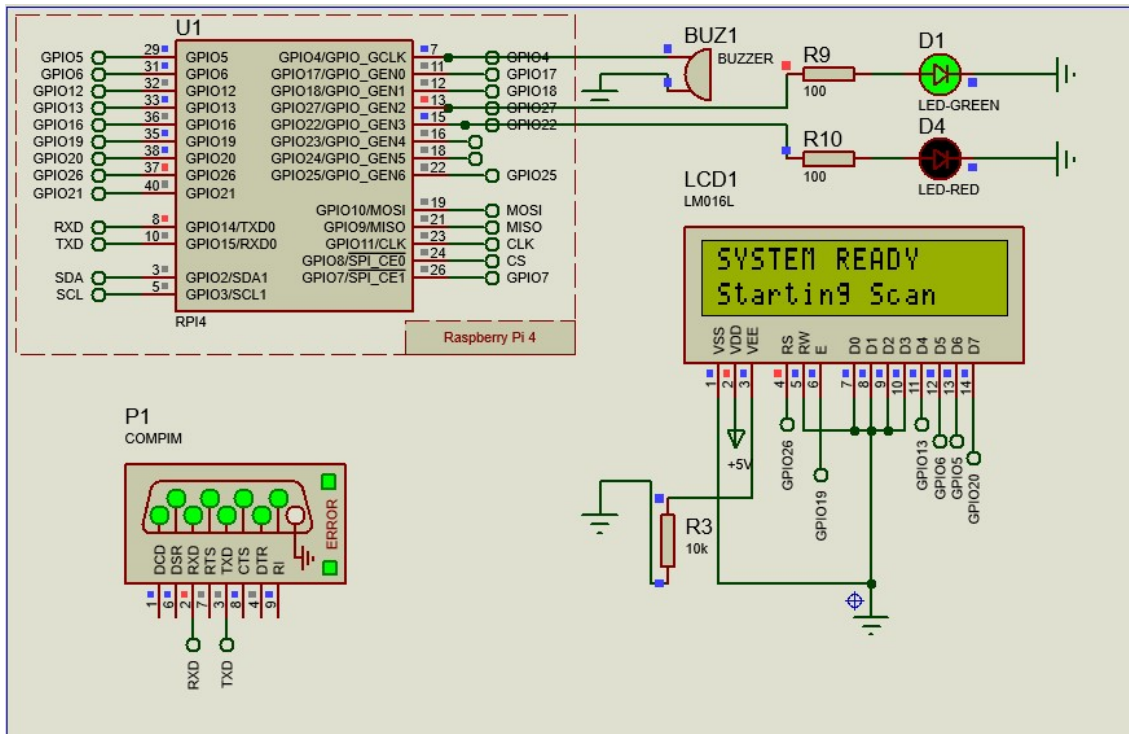


Figure 4.6: System after Synchronization with LCD

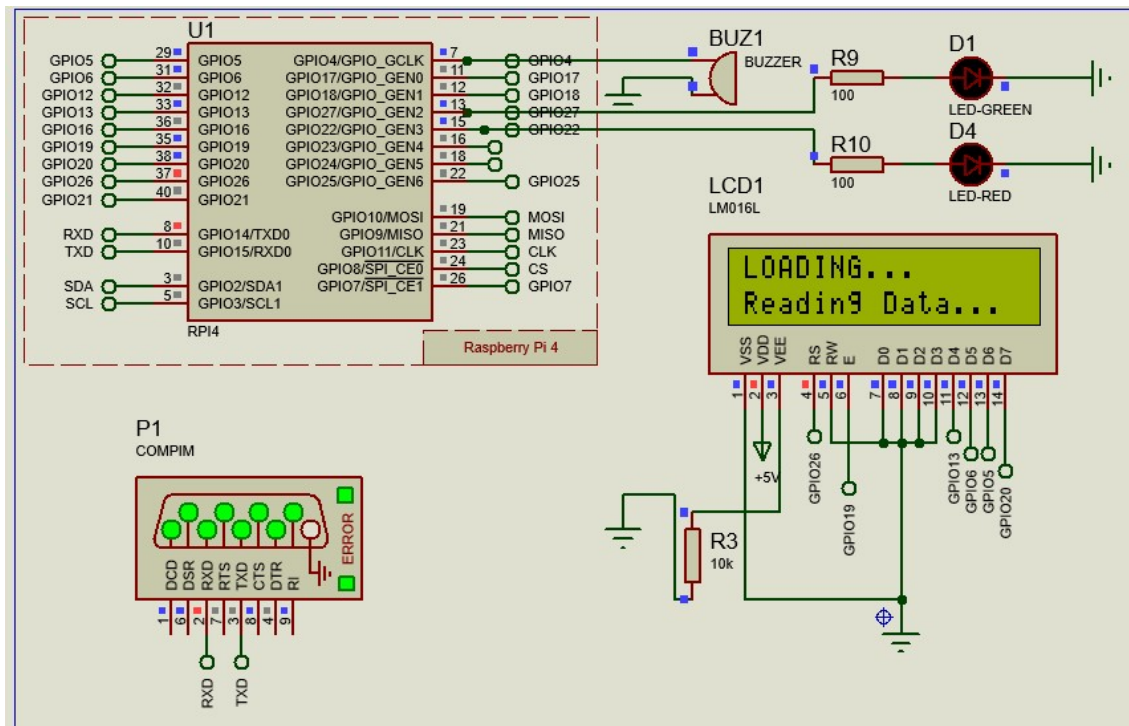


Figure 4.7: System when it loads the dataset

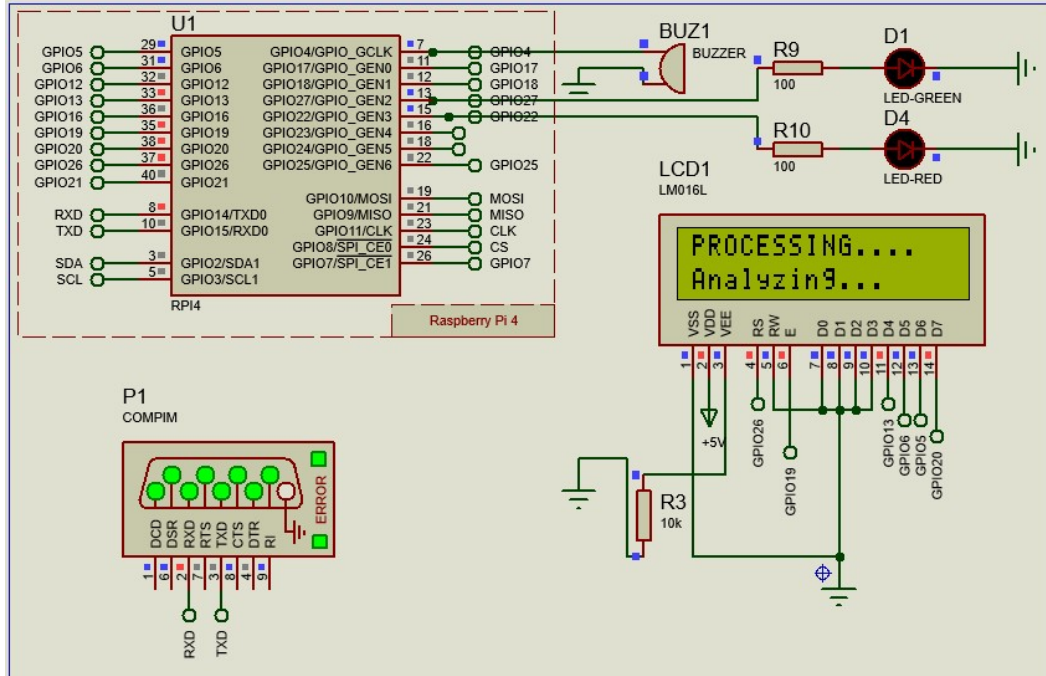


Figure 4.8: system when it processes the Dataset

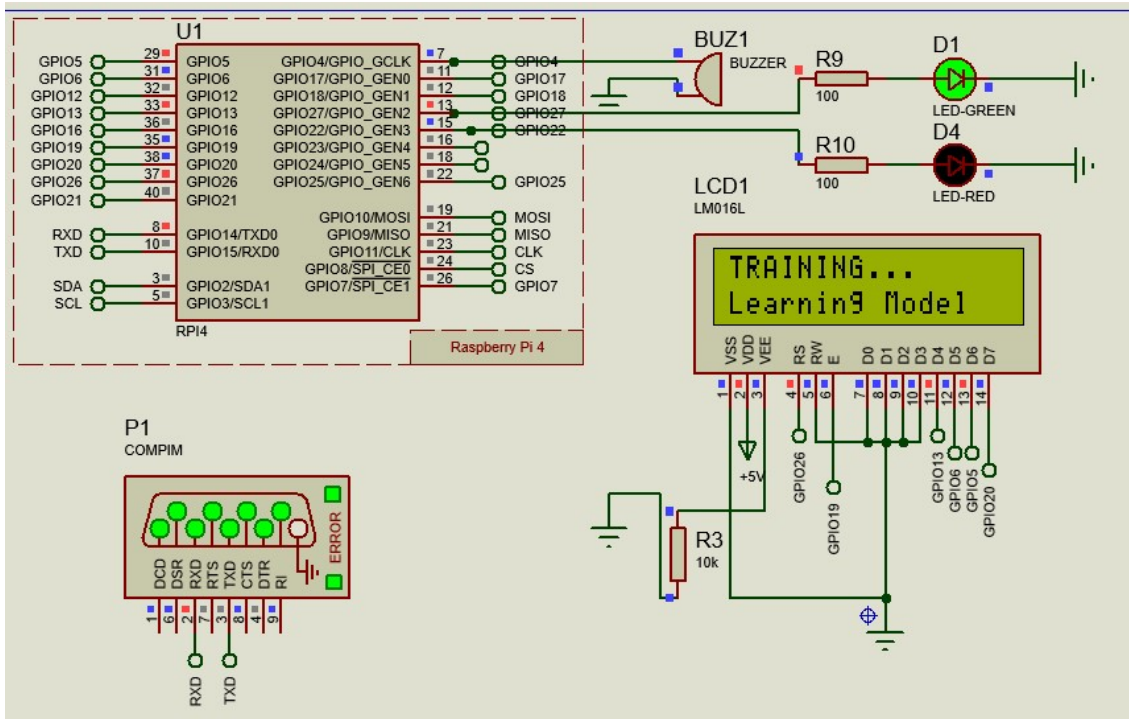


Figure 4.9: System when training

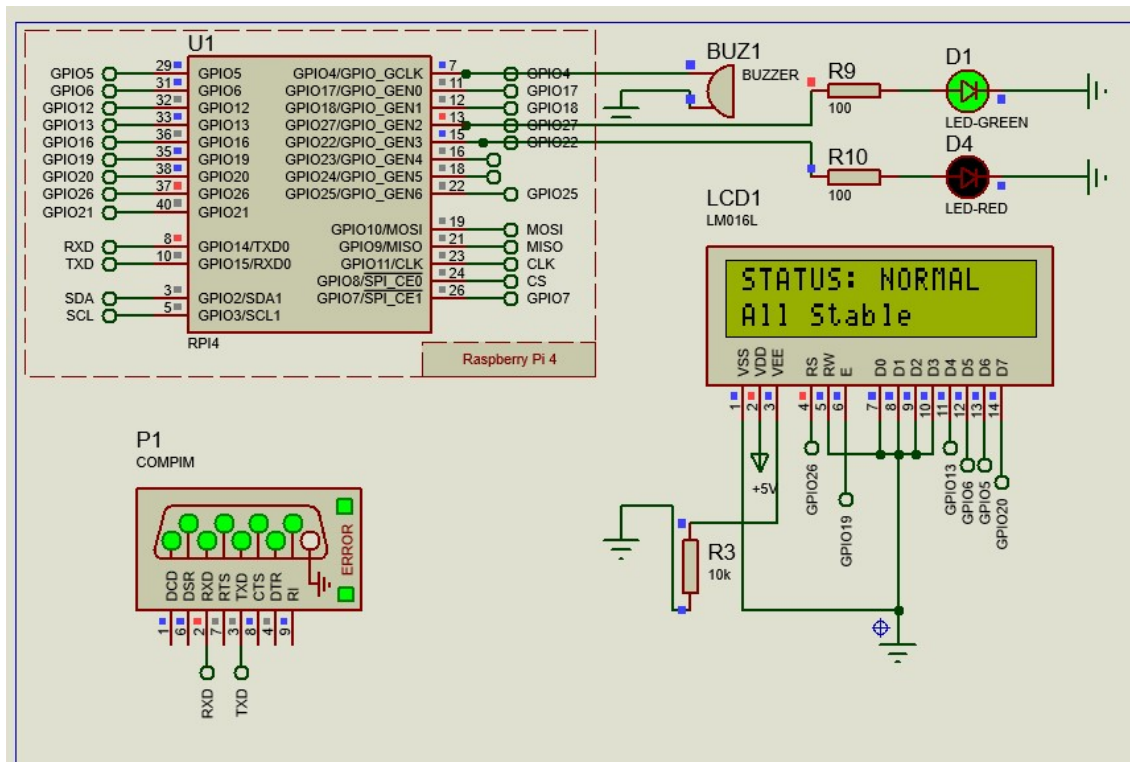


Figure 4.10: System after training and it outputs results, when there is No Seizure Normal

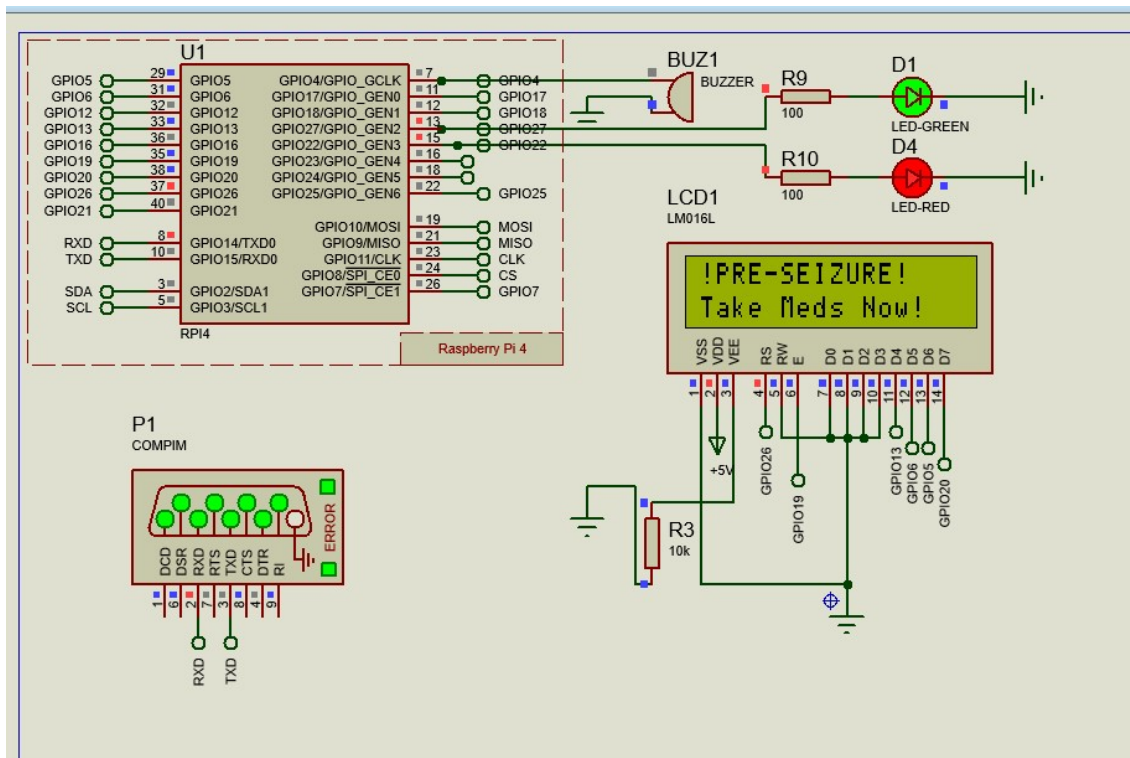


Figure 4.11: System before Seizure Occurs PRE SEIZURE

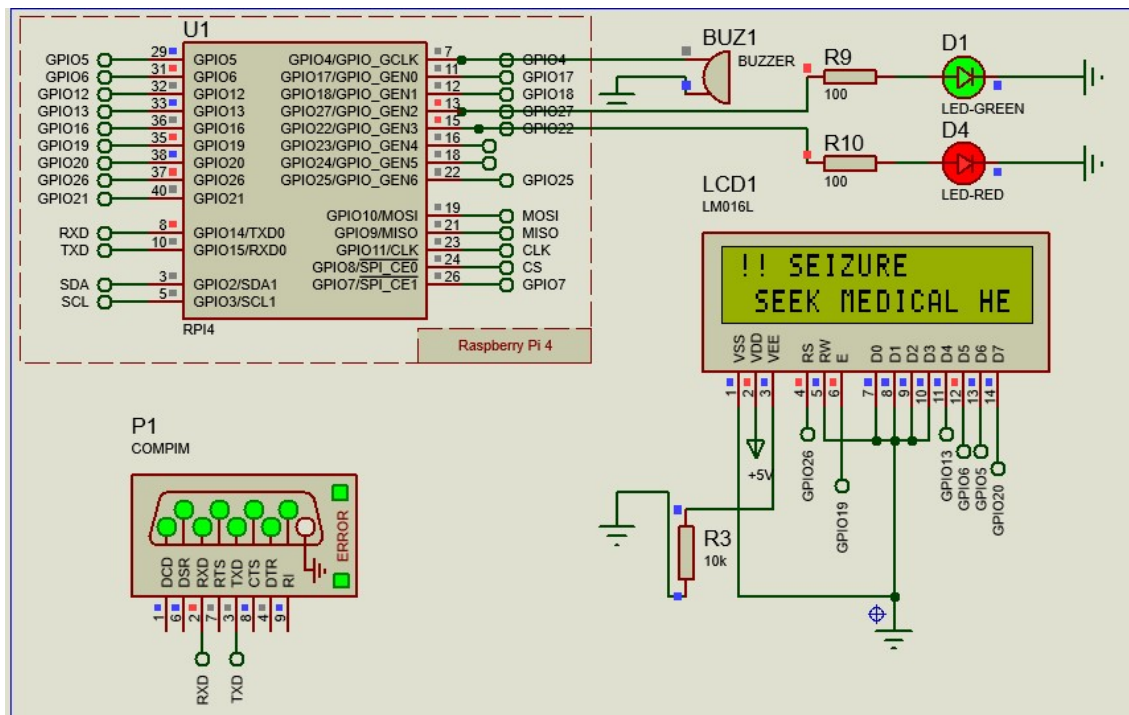


Figure 4.12: System when the Seizure Occurs SEIZURE

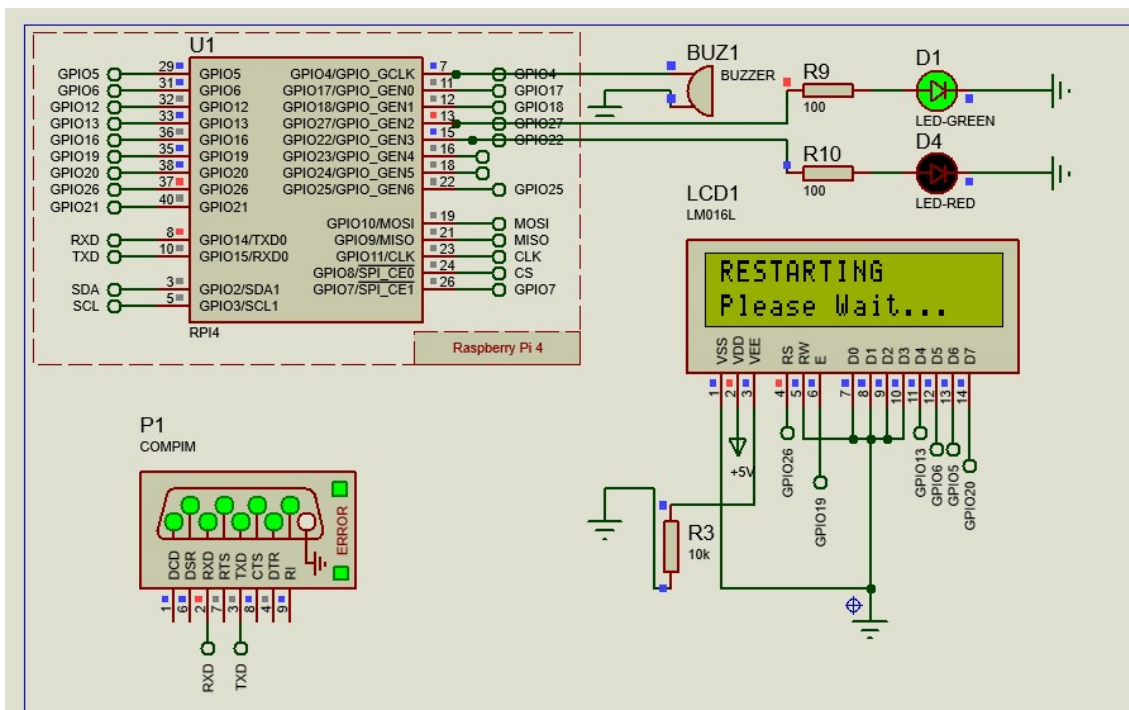


Figure 4.13: System after the seizure detection is done and it restarts RESTARTING

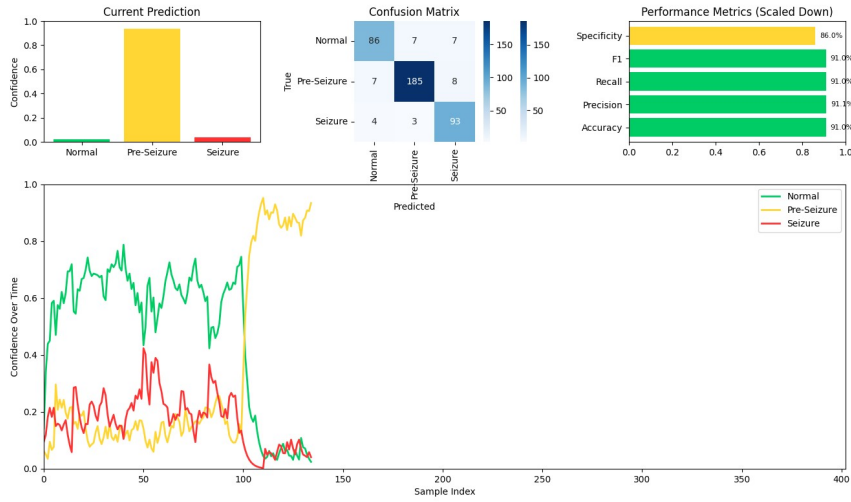


Figure 4.14: Simulation Circuit of the Seizure detection circuit

4.4 Monitoring System

4.4.1 Dashboard Overview

The NeuroWatch dashboard was implemented using the Streamlit framework in Python and serves as the primary clinical interface through which medical staff can monitor patient EEG states in real time. The dashboard reads from a shared JSON bridge written continuously by the Raspberry Pi runtime and refreshes every five seconds. I designed the interface around five tabs — Ward Overview, Patient Detail, Ward Map, Event Log, and Model Metrics — each serving a distinct monitoring purpose, with a persistent sidebar visible at all times regardless of which tab is active.

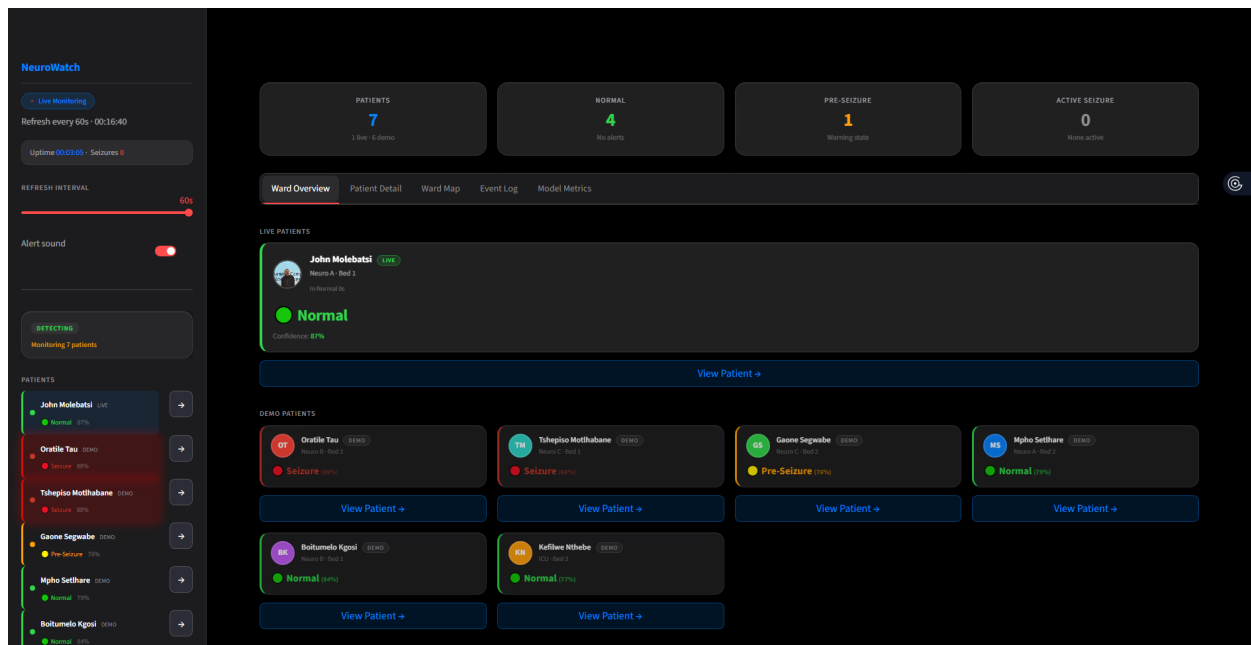


Figure 4.15: NeuroWatch dashboard home view showing the ward overview layout used for real-time clinical monitoring.

4.4.2 Sidebar and Global Controls

The sidebar on the left of the screen is always visible and provides the doctor with an at-a-glance summary of every patient being monitored. At the top of the sidebar the NeuroWatch logo and a pulsing *Live Monitoring* badge confirm that the system is actively running. Directly below this, an uptime counter shows how long the current session has been active alongside a cumulative count of seizure events detected during that session.

Each patient is listed in the sidebar as a labelled entry showing their name, a coloured dot indicating their current EEG state (green for Normal, amber for Pre-Seizure, and red for Seizure), the confidence percentage reported by the classifier, and a small badge identifying whether they are a live or demo patient. Patients are sorted so that the most urgent cases always appear at the top. The doctor can click the arrow button next to any patient name to jump directly to that patient’s detail view. If a live patient enters a Seizure state the corresponding sidebar entry begins to flash with a blood-red blink animation, drawing immediate attention even when the clinician is looking at a different part of the screen.

Below the patient list the sidebar displays a scrollable alert history showing the eight most recent state-transition events. Each alert entry shows the patient name, ward and bed, the new state, the confidence score, the time the transition occurred, and, where applicable, a link to the patient’s GPS location in Google Maps. A toggle switch allows the doctor to enable or disable an audible beep alarm that sounds whenever a live patient transitions into a Seizure state. A refresh-interval slider lets the clinician choose how frequently the dashboard polls for new data, with options ranging from two seconds up to sixty seconds.

4.4.3 Ward Overview Tab

When the dashboard first opens the doctor lands on the Ward Overview tab. This tab gives a bird’s-eye view of every patient currently being monitored. At the very top of the main panel a row of four summary cards shows the total number of patients, how many are currently Normal, how many are in a Pre-Seizure warning state, and how many have an active Seizure. If any live patient is seizing the Active Seizure card itself begins to flash red, and a full-width coloured banner appears above the cards stating the patient’s name and instructing the clinician to respond immediately.

Below the summary cards the tab is divided into a *Live Patients* section and a *Demo Patients* section. The live patient card is rendered at full width and displays a circular avatar, the patient’s name, their ward and bed number, the current EEG classification in large coloured text, and the confidence percentage. The demo patient cards are arranged in a grid of four columns and show the same information in a more compact form. Every card has a *View Patient* button that the doctor can click to navigate directly to the Patient Detail tab with that specific patient already selected, avoiding the need to manually locate the patient in the sidebar.

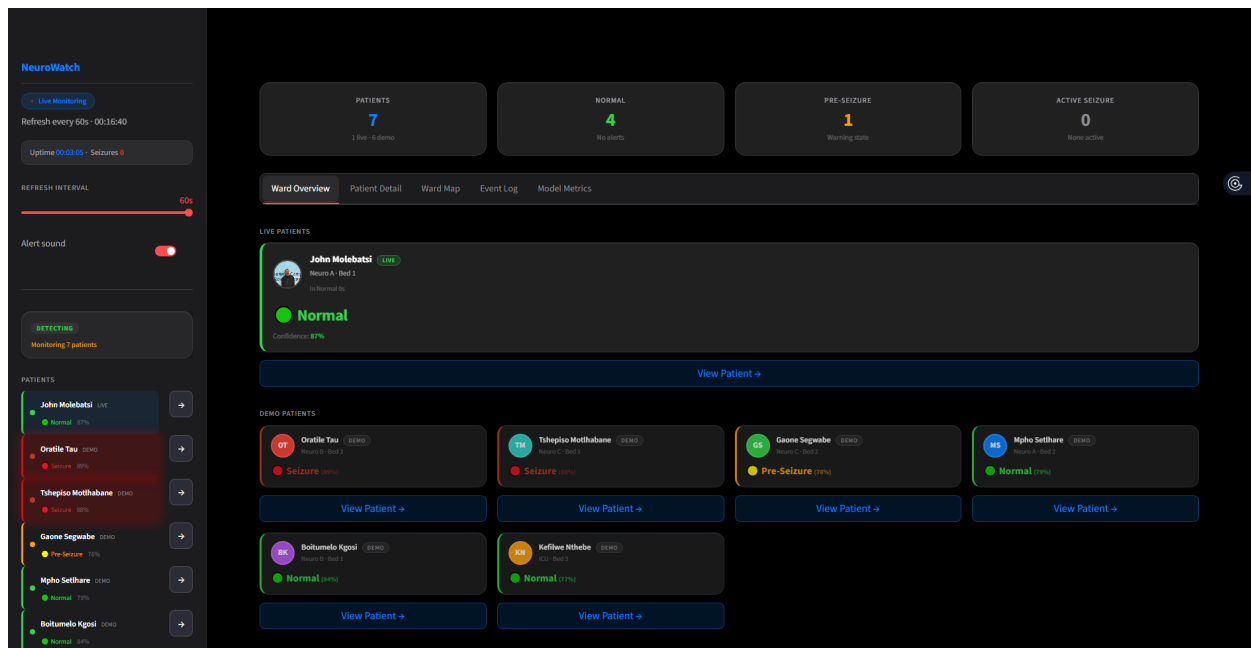


Figure 4.16: Ward Overview behaviour during a seizure event, showing the visual escalation used to draw clinician attention.

4.4.4 Patient Detail Tab

The Patient Detail tab is the most information-dense screen in the dashboard and is where the doctor would spend the most time during an active monitoring session. At the top of the tab a header card displays the selected patient’s name, their live or demo designation, ward, bed, current EEG state in large bold text, and the classifier’s confidence. For a patient currently classified as Seizure the entire header pulses red.

The main content area is split into two equal columns. On the right side an interactive three-dimensional model of a human brain is rendered and responds in real time to the patient’s current EEG state. The brain surface is coloured using a heatmap derived from the 10–20 electrode positions: during a Normal classification the surface glows with a calm blue tone, during Pre-Seizure the frontal regions glow amber and blink slowly, and during an active Seizure the temporal lobes ignite in a sharp red-orange colour and blink rapidly to indicate the modelled seizure focus. The doctor can drag the mouse to rotate the brain, scroll to zoom in or out, and examine any angle to understand which cortical regions are showing elevated activity.

On the left side of the tab the doctor can read four quantitative displays stacked vertically. The first is a donut chart showing the proportion of time the patient has spent in each of the three states since monitoring began, giving a quick sense of how frequently the patient has been transitioning. Below the donut chart four horizontal band-power bars labelled Delta, Theta, Alpha, and Beta show the relative spectral power of the patient’s most recent EEG segment in each frequency band, providing the underlying feature values that drove the current classification. Below the band-power bars a line chart plots the state trajectory over the most recent sixty seconds, with each data point coloured according to its state, so the doctor can immediately see whether the patient has been stable or oscillating. A second line chart below that shows how the raw band-power values have evolved over the same sixty-second window, and a final filled area chart at the bottom shows the classifier confidence as a per-

centage over the same period, allowing the clinician to judge whether the model’s certainty has been consistent or variable.

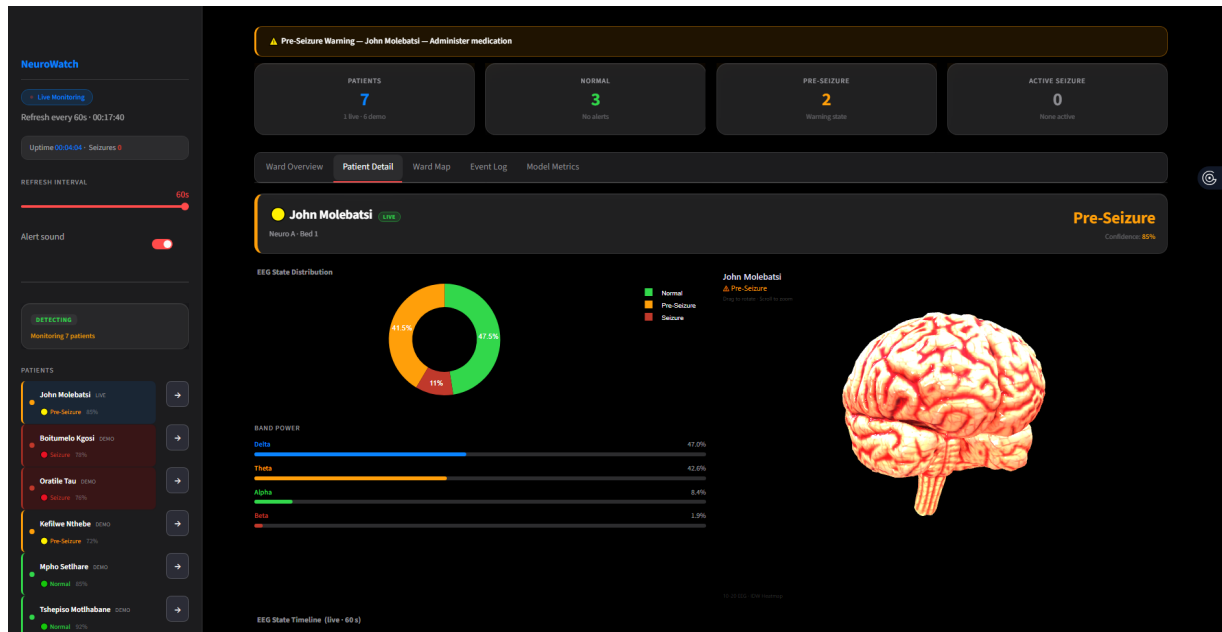


Figure 4.17: Patient Detail tab showing the per-patient diagnostic layout used for live monitoring and interpretation.



Figure 4.18: Feature-extraction and band-power visual components presented in the Patient Detail view.

4.4.5 Ward Map Tab

The Ward Map tab gives the doctor a geographic view of where every patient is physically located within Botswana and which hospitals are nearby. The top section of the tab shows a full-width interactive map with a coloured circular marker for each patient positioned at their registered GPS coordinates. The colour of each marker matches the patient’s current EEG state so that the clinician can immediately see whether any geographically remote patient is in distress. Dashed lines connect each patient marker back to the operator’s reference location in Palapye, and the map also displays diamond-shaped markers for hospitals and clinics across the country. Clicking any patient marker opens a small popup showing the patient’s name, ward, current state, confidence, and the straight-line distance from Palapye. Clicking a hospital marker shows the facility name, type, and its distance from the operator.

Below the overview map a row of distance cards shows each patient ranked by their proximity to the operator, with the straight-line distance in kilometres displayed prominently in blue. This allows the doctor to quickly identify which patients are nearby and which would require longer travel or coordination with remote facilities.

In the lower section of the Ward Map tab the doctor can select any individual patient from a dropdown menu to see a zoomed map centred on that patient’s exact location. This zoomed map shows the nearest hospitals ranked by distance from the patient, each with a popup describing the facility type, its distance from the patient, and its distance from the operator. A footer bar beneath the map displays the patient’s GPS coordinates in decimal degrees, the distance from Palapye, and a *Google Maps* button that opens the exact location in a browser so that the doctor can get turn-by-turn navigation if needed.

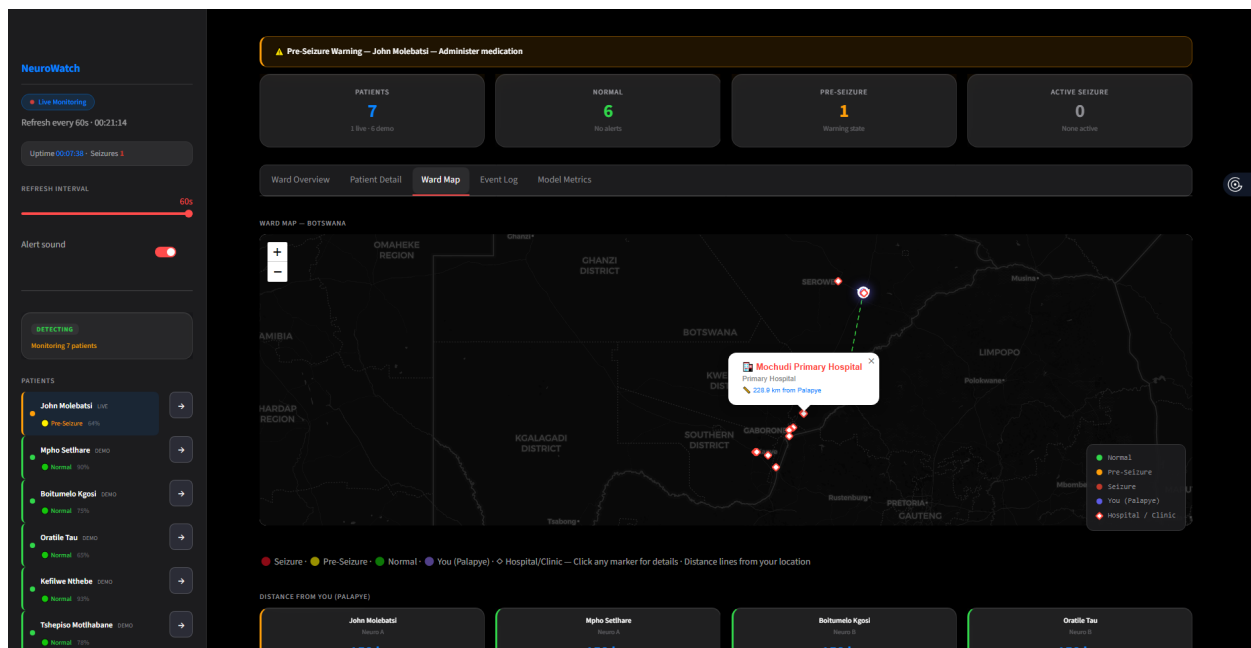


Figure 4.19: Ward Map overview showing patient locations, hospitals, and distance context across Botswana.

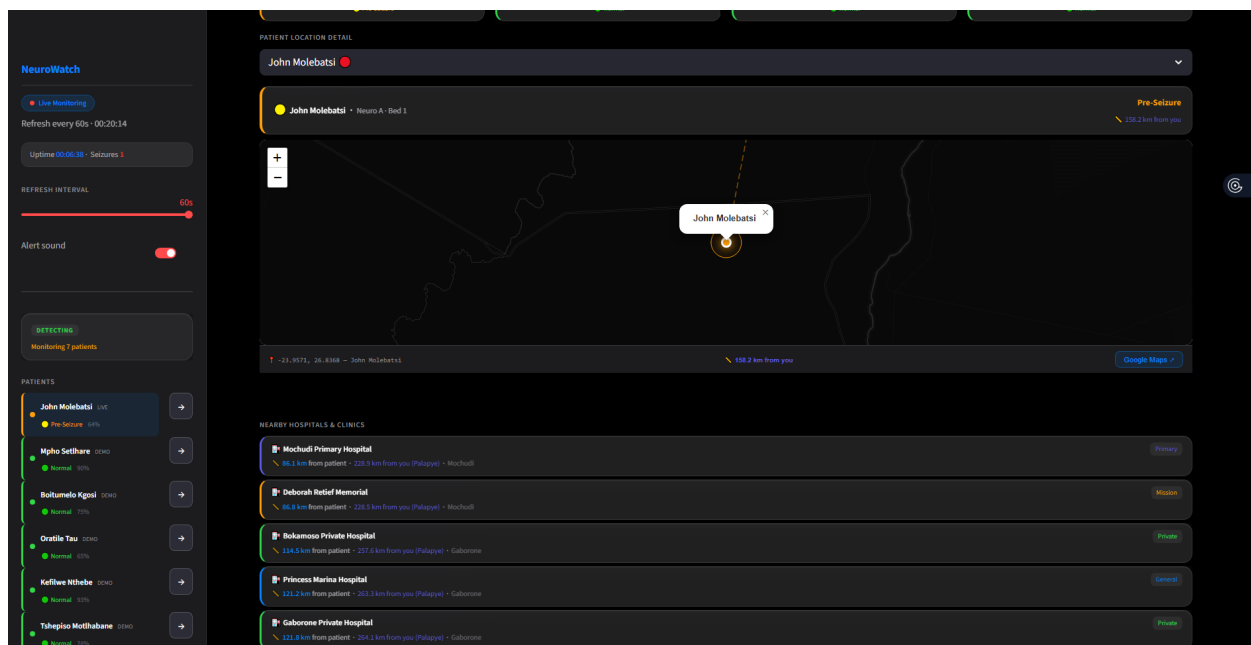


Figure 4.20: Zoomed ward-map view for a selected patient with nearby facilities and exact location context.

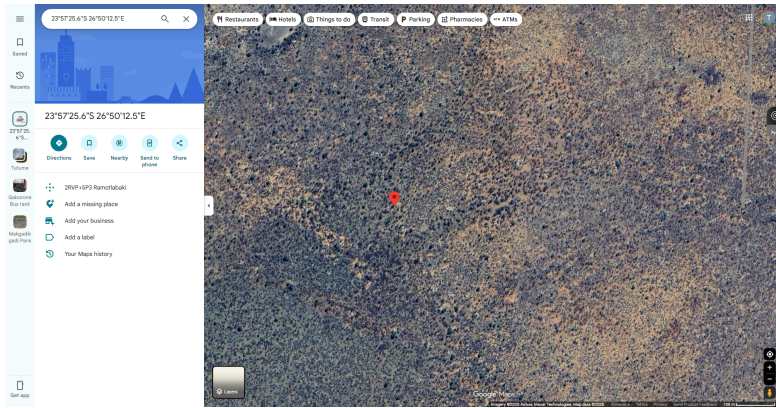


Figure 4.21: Zoomed ward-map view for a selected patient with nearby facilities and exact location context.

4.4.6 Event Log Tab

The Event Log tab provides the doctor with a detailed chronological record of every classification the system has produced for any selected patient. The doctor first uses the dropdown at the top of the tab to choose which patient’s history to inspect. A set of filter controls then allows the log to be narrowed by state type using a multi-select picker, limited to a chosen number of the most recent rows using a slider, or restricted to seizure events only using a checkbox.

The resulting table displays each record’s timestamp, the classified state, the model confidence, and for live patients the four band-power feature values that the classifier received as input. An SMS-sent flag column indicates whether a Twilio notification was dispatched for that record. A summary line beneath the table reports the total number of records, the absolute count of Seizure and Pre-Seizure events, and the overall seizure rate as a percentage, giving the doctor a quick statistical summary of the session. The doctor can download the complete unfiltered history for the selected patient as a CSV file using the *Download CSV* button, which is useful for handing records over to a neurologist or archiving them in a clinical information system.

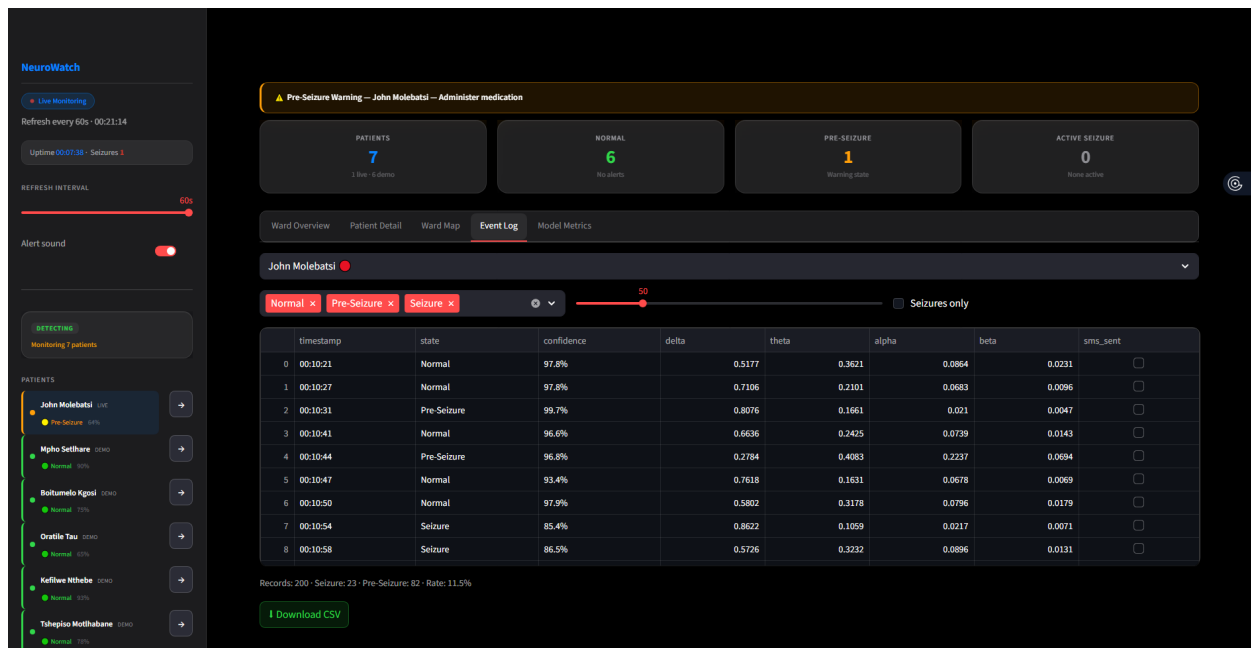


Figure 4.22: Event-history visualisation used to inspect previous patient states and review state-transition patterns over time.

4.4.7 Model Metrics Tab

The Model Metrics tab allows the doctor or a supervising engineer to verify the performance of the underlying classification model before trusting the system’s outputs in a clinical setting. Six metric cards at the top of the tab display the model’s accuracy, precision, recall, F1 score, specificity, and seizure-specific recall computed against the full set of EEG segments loaded at startup. Each card is coloured green if the metric exceeds ninety percent, amber if it falls between seventy and ninety percent, and red if it falls below seventy percent, so the clinician can see at a glance whether the model meets an acceptable performance standard.

A horizontal bar chart below the cards repeats the same six metrics in graphical form, and a confusion matrix heatmap shows the exact count of correct and incorrect classifications for each of the three classes, allowing a more experienced user to assess whether any particular error type — such as missing a seizure or misclassifying a normal segment — is more prevalent than another. A caption line at the top of the tab records the timestamp at which the model was last evaluated, the dataset it was trained on, and the classifier architecture, providing the traceability that a clinical or regulatory review would require.

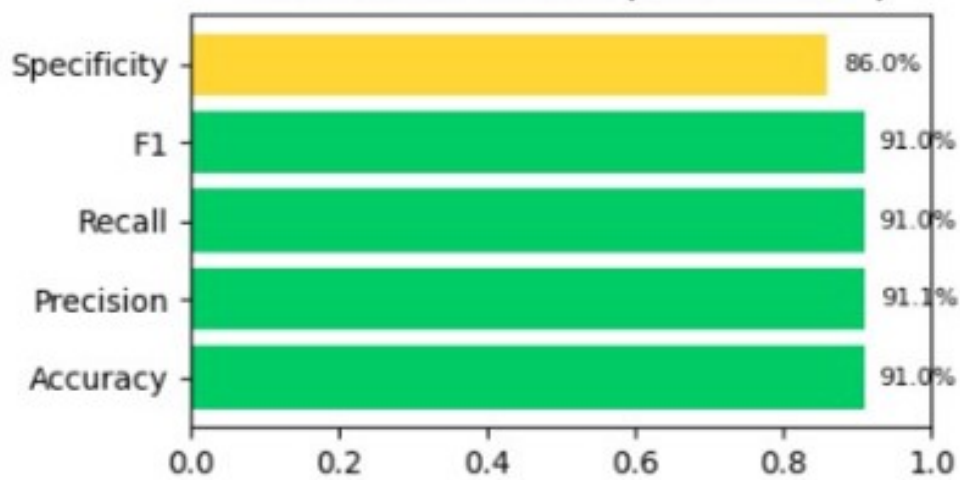


Figure 4.23: Dashboard model-metrics visualisation summarising the classifier performance measures available to clinicians and engineers.

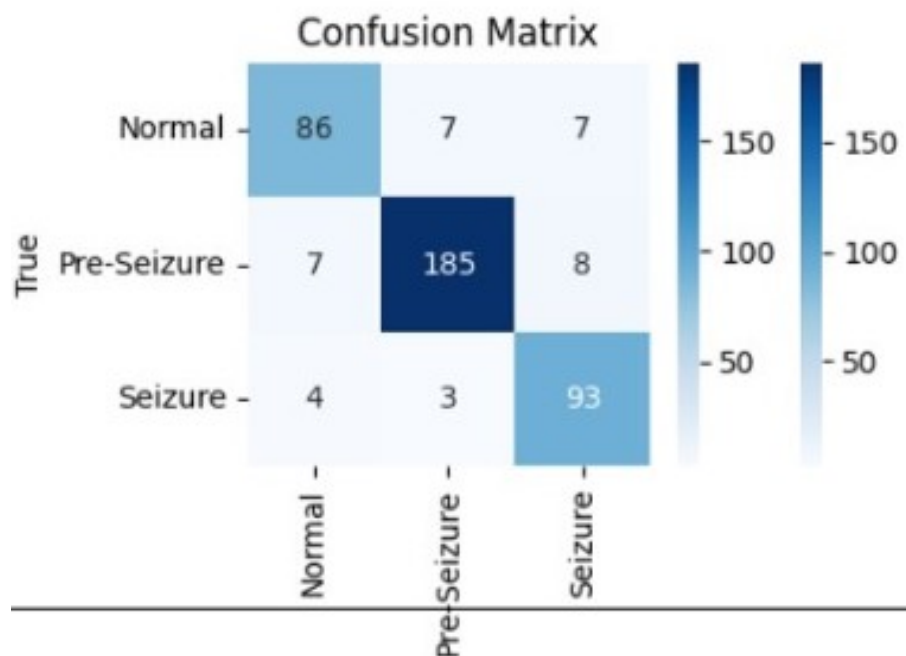


Figure 4.24: Confusion matrix view used in the Model Metrics tab to inspect classification errors across the three EEG states.

4.5 Patient User-Interface(UI)

To give the patient direct visibility into their own monitoring status, a separate patient-facing interface was developed using Python’s tkinter framework. The interface adopts a smartwatch-inspired dark-theme aesthetic and presents the classifier output in plain English rather than clinical terminology, using three simple states — Normal, Heads-up, and Alert — so the patient requires no medical background to interpret their status. A large circular activity ring dominates the screen, filling proportionally to the classifier confidence and changing colour from green to amber to red as the detected state escalates, with the percentage and plain-English label displayed inside the ring. Below the ring a short action message updates accordingly, reading instructions such as *Please stay calm — Contact Doctor / Emergency* during an Alert state. Supporting panels beneath the message show a consecutive normal-reading streak counter, a last-checked timestamp, a dot-history strip summarising recent classification outcomes in colour-coded sequence, and the patient’s registered emergency contact name. A small pulsing green indicator at the bottom of the screen confirms that live monitoring is active. Because the interface reads from the same shared JSON bridge as the clinician dashboard, both views always reflect identical underlying classification data, and the patient-facing UI can run on any bedside device on the same network without additional hardware.

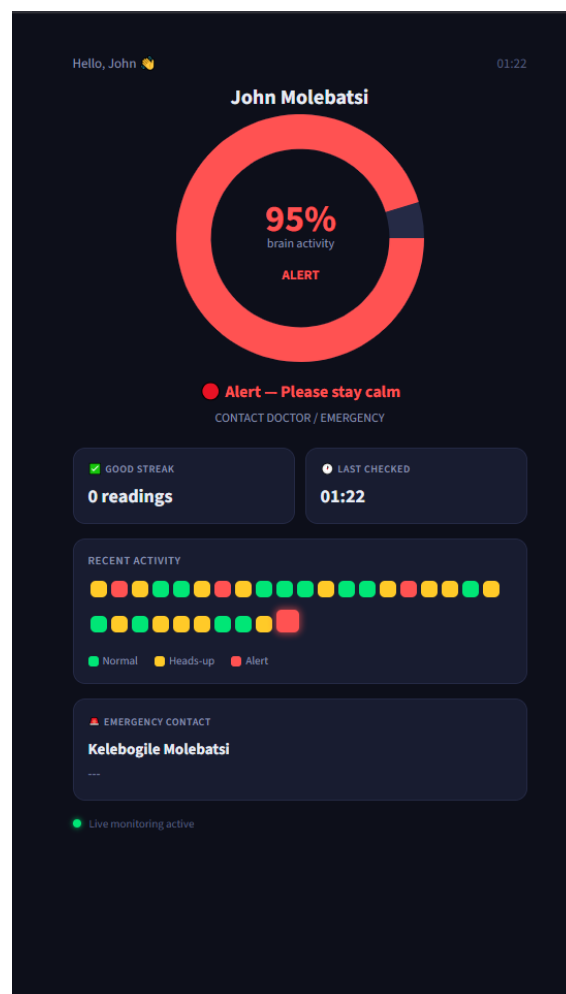


Figure 4.25: Patient-facing user interface showing the smartwatch-inspired status view used to communicate EEG state and guidance in plain language.

4.6 Simulation Results

4.6.1 Overview of System operation

The developed machine learning-based predictive framework integrates three distinct sub-systems operating in synchronized coordination: the Python-based machine learning model running on a desktop environment, the virtual serial communication interface enabling cross-platform data exchange, and the Raspberry Pi 4 embedded controller managing the LCD display and alert mechanisms. The system operates in a continuous cycle beginning with initialization, progressing through data loading where EEG files from the Bonn University dataset are read and processed, followed by feature extraction using bandpass filtering and power spectral density analysis to derive delta, theta, alpha, and beta frequency band features. These compact four-dimensional feature vectors are then fed into a hybrid classification pipeline combining Support Vector Machine and Random Forest models trained using an 80-20 train-test split. Real-time predictions classify each EEG segment into one of three states: Normal (pred=0), Pre-Seizure (pred=1), or Seizure (pred=2), with corresponding notifications transmitted via the virtual serial interface using compressed opcodes (NORM, WARN, SEIZ) to maintain communication efficiency. The Raspberry Pi controller receives these opcodes and translates them into user-facing outputs: LCD text messages providing state information, color-coded LED signals (continuous green blink indicating system operation, intermittent red blink for Pre-Seizure warnings, rapid red blink for Seizure alerts), and auditory buzzer activation during critical states. This multi-modal feedback strategy ensures that alerts reach users through multiple sensory channels, enhancing reliability in real-world monitoring scenarios. The system completes its processing pipeline by computing comprehensive validation metrics including confusion matrices, accuracy, precision, recall, F1 scores, and specificity measures, before restarting the cycle to simulate continuous monitoring operation.

4.7 Confusion Matrix and Performance Metrics

4.7.1 Confusion Matrix

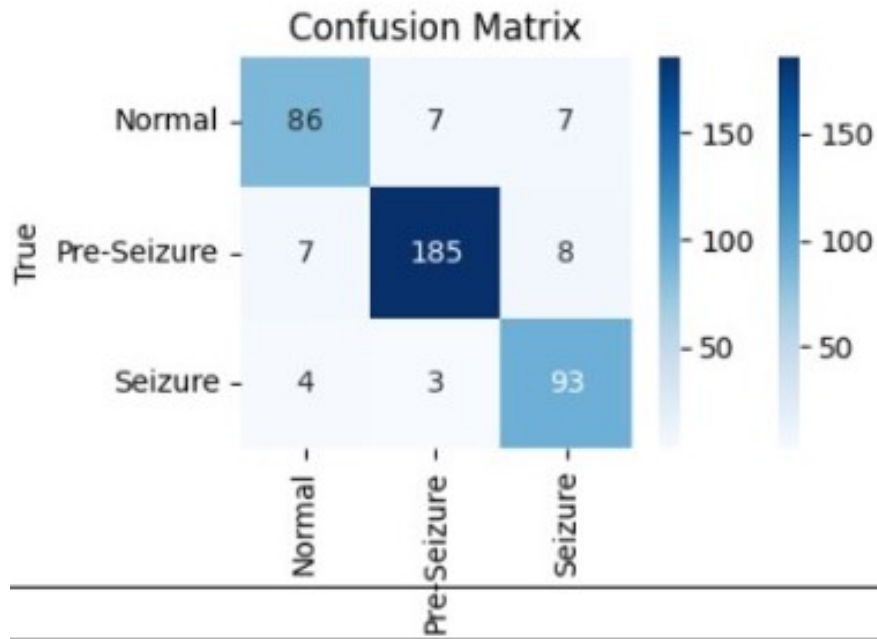


Figure 4.26: Confusion Matrix of the Model

The confusion matrix provides a detailed view of the model's predictions across the three-state classification:

- **Normal:** Out of 100 Normal samples, 86 were correctly classified, while 7 were misclassified as Pre-Seizure and 7 as Seizure. This indicates a small tendency toward false alarms, possibly due to overlapping EEG characteristics during transitional states or the presence of normal physiological artifacts that share spectral features with seizure-related patterns. The 86% true positive rate for Normal states demonstrates that the system maintains good specificity despite its bias toward sensitivity, correctly identifying the majority of stable, non-epileptic brain activity. The balanced distribution of errors between Pre-Seizure and Seizure (7 each) suggests that when Normal states are misclassified, the model does not systematically favor one particular error type.
- **Pre-Seizure:** The model performs exceptionally well in detecting Pre-Seizure episodes, correctly classifying 185 out of 200 samples, achieving 92.5% recall. Only 7 samples were confused with Normal and 8 with Seizure. This suggests robust sensitivity to early seizure-related EEG patterns that emerge during the transitional phase preceding full seizure onset. The high performance on this class is particularly significant because Pre-Seizure detection represents the system's primary value proposition for early intervention, providing patients and caregivers with a critical time window for preventive actions such as medication administration or safety positioning. The relatively low confusion with Normal states (7 instances) indicates that the feature extraction pipeline effectively captures subtle changes in brain activity that distinguish pre-ictal patterns from baseline conditions.
- **Seizure:** Seizure events are identified with high effectiveness, with 93 correct classifications out of 100 samples, achieving 93% recall. Misclassifications are minimal: 4

samples incorrectly labeled as Normal and 3 as Pre-Seizure, demonstrating the model’s strong ability to detect seizure-specific patterns once the full ictal phase develops. The very low false negative rate (7%) ensures that genuine seizure events are rarely missed, which is critical for patient safety and for maintaining user trust in the monitoring system. The few errors that occur primarily involve confusion with Pre-Seizure states (3 instances), which is understandable given that some seizures may be captured at their onset phase where characteristics overlap with the pre-ictal period.

4.8 Performance Metrics Explanation

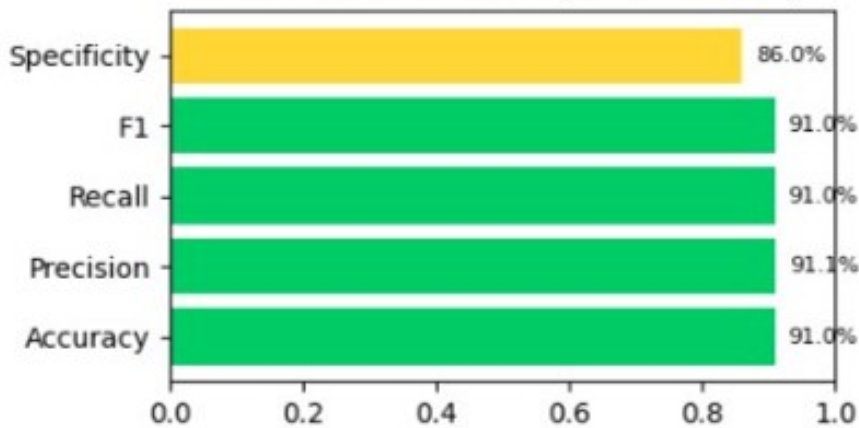


Figure 4.27: Performance Metrics of the Model

- Accuracy (91.0%):** Reflects the overall proportion of correctly predicted samples across all three classes. The high value demonstrates consistent performance across the dataset and places this framework within the range of clinically viable seizure detection systems reported in published literature (typically 85-95% on the Bonn dataset). This accuracy level represents a carefully calibrated balance between model complexity and practical deployability, ensuring the system remains computationally efficient for embedded platforms while providing clinically useful predictions.
- Precision (91.1%):** Indicates that when the model predicts a class (especially Pre-Seizure and Seizure), it is correct 91.1% of the time. This high precision is useful for reducing false alarms and maintaining user trust, as only approximately 9% of positive predictions are incorrect. In the context of continuous monitoring where dozens of predictions occur daily, this precision level translates to manageable numbers of false positives that users can tolerate without developing alarm fatigue—a critical consideration for medical monitoring devices where excessive false alarms lead users to ignore or disable alerts.
- Recall (91.0%):** Shows that the model successfully retrieves 91% of all actual positive cases (Pre-Seizure and Seizure events), which is essential in medical early-warning systems where missing genuine events carries serious safety consequences. The high recall ensures that the vast majority of seizure-related states are detected, enabling timely intervention and reducing risks of injury, prolonged seizures, or sudden unexpected death in epilepsy (SUDEP). The class-specific recalls—86% for Normal, 92.5% for Pre-Seizure,

and 93% for Seizure—demonstrate differentiated performance with particularly strong detection of the critical Pre-Seizure and Seizure states.

- **F1-Score (91.0%):** Balances precision and recall, confirming strong performance across both dimensions even with potential class imbalance. The F1 score’s harmonic mean formulation penalizes situations where one metric is high at the expense of the other, ensuring that the 91% value reflects genuinely balanced performance rather than artificially inflated accuracy from bias toward the majority class. This balance is important because naive strategies could achieve high recall by predicting positive for every sample (detecting all seizures but with terrible precision) or high precision by predicting positive only for obvious cases (missing many subtle seizures).
- **Specificity (86.0%):** Measures the ability to correctly identify non-events, particularly Normal states when seizure-related conditions are truly absent. The value of 86% indicates that the system correctly rejects 86% of true negative cases, though it is slightly lower than the other metrics. This suggests occasional false positives where Normal states are incorrectly flagged as seizure-related, but the value remains acceptable for biomedical applications, especially considering that the conservative bias toward sensitivity (catching real seizures) over specificity (avoiding false alarms) is appropriate for safety-critical medical monitoring where the cost of missed detections exceeds the cost of false warnings.

4.9 Discussion

The developed machine learning-based seizure detection framework successfully fulfills all three project objectives while maintaining explicit optimization for deployment in low-resource settings. The system achieves 91% accuracy through a hybrid SVM-Random Forest classification approach that balances algorithmic sophistication against computational constraints, processing EEG signals through efficient bandpass filtering and power spectral density feature extraction that reduces thousands of time-domain samples to just four frequency-band power values. The three-state classification (Normal, Pre-Seizure, Seizure) provides clinically meaningful granularity beyond simple binary seizure/non-seizure detection, with the Pre-Seizure class achieving particularly strong 92.5% recall that enables early intervention during the critical transitional phase. The confusion matrix analysis reveals that misclassifications primarily occur at physiologically plausible boundaries between adjacent states, with 86% Normal detection, 92.5% Pre-Seizure detection, and 93% Seizure detection demonstrating robust performance across all classes. The model’s balanced precision (91.1%) and recall (91.0%) profile minimizes both false alarms that could cause user distrust and missed detections that could compromise patient safety, while the 86% specificity reflects an appropriate conservative bias toward sensitivity in a safety-critical medical application.

The model demonstrates several key strengths that validate its suitability for seizure monitoring applications:

- **Reliable classification of seizure-related EEG patterns:** The 91% overall accuracy and balanced performance across precision, recall, and F1 metrics confirm that the hybrid ensemble approach effectively discriminates between Normal, Pre-Seizure, and Seizure states using frequency-domain power features.
- **Strong ability to detect early Pre-Seizure signals:** The 92.5% recall for the Pre-Seizure class, which represents the highest performance among the three states,

demonstrates that the system excels at identifying subtle precursor patterns that emerge before full seizure onset. This capability is crucial for timely intervention, potentially enabling preventive actions during the critical warning window.

- **Low misclassification across classes:** The confusion matrix reveals that errors are relatively infrequent and tend to occur at physiologically plausible boundaries between adjacent states rather than as dramatic mistakes. The total misclassification rate of 9% is distributed fairly evenly across error types, indicating no systematic bias toward particular failure modes.
- **High precision and recall indicating balanced performance:** The near-identical values for precision (91.1%) and recall (91.0%) demonstrate that the model avoids the common pitfall of optimizing one metric at the expense of the other. This balance ensures both that alerts are trustworthy (high precision) and that genuine events are caught (high recall).

4.10 Recommendations

Based on the results and identified limitations, several recommendations emerge for advancing this seizure detection framework toward clinical deployment and enhanced performance:

- **Dataset Expansion:** The system should be validated on additional publicly available datasets including CHB-MIT Scalp EEG Database, Siena Scalp EEG Database, and Temple University Hospital EEG Corpus to assess generalization beyond the Bonn dataset. Expanding the training data with these diverse sources would expose the model to broader EEG variability patterns across different epilepsy syndromes, age groups, and recording conditions, potentially improving the current 86% specificity and overall robustness.
- **Validation on Diverse EEG Sources:** Cross-dataset validation testing the model trained on Bonn data against independent datasets would reveal performance under diverse recording conditions, seizure types, and patient populations, identifying any dataset-specific biases in the current feature extraction pipeline. Additionally, prospective data collection from ambulatory patients wearing portable EEG devices in daily-life conditions would test robustness against real-world artifacts including motion, electrode displacement, muscle contamination, and environmental electrical noise that are minimized in laboratory recordings.

.1 Source Code

.1.1 LCD Controller

Listing 1: Raspberry Pi LCD Controller Implementation

```
# -----  
# Raspberry Pi 4 – OPTIMIZED LCD Controller with Fixed Timing  
# Resolves garbled text issues with proper HD44780 timing  
# -----  
  
import RPi.GPIO as GPIO  
from time import sleep, time  
import threading  
import os  
  
# ----- PIN CONFIGURATION -----  
LCD_RS = 26  
LCD_E = 19  
LCD_D4 = 13  
LCD_D5 = 6  
LCD_D6 = 5  
LCD_D7 = 20  
GREEN_LED = 27  
RED_LED = 22  
BUZZER = 4  
  
LCD_WIDTH = 16  
LCD_CHR = True  
LCD_CMD = False  
LCD_LINE_1 = 0x80  
LCD_LINE_2 = 0xC0  
  
# HD44780 Timing Constants (critical for preventing garbled text)  
E_PULSE = 0.0005 # 500 microseconds (increased from 450)  
E_DELAY = 0.0005 # 500 microseconds  
INIT_DELAY = 0.005 # 5ms for initialization commands  
CMD_DELAY = 0.002 # 2ms for regular commands  
CHAR_DELAY = 0.0001 # 100 microseconds between characters  
  
# Virtual Serial via shared file  
VIRTUAL_SERIAL_RX = "C:/Users/thebe/AppData/Local/Temp/vsm_serial_rx.txt"  
VIRTUAL_SERIAL_TX = "C:/Users/thebe/AppData/Local/Temp/vsm_serial_tx.txt"  
LCD_REFRESH_RATE = 0.3  
GREEN_BLINK_RATE = 0.5  
  
# -----  
# STATE MACHINE  
# -----  
  
class SystemState:  
    BOOT = 0
```

```

INIT = 1
LOADING = 2
PROCESSING = 3
TRAINING = 4
READY = 5
NORMAL = 6
PRE_SEIZURE = 7
SEIZURE = 8
RESTARTING = 9
WAITING_MODEL = 10

# -----
# OPTIMIZED LCD FUNCTIONS WITH PROPER TIMING
# -----
def lcd_toggle_enable():
    """Toggle_enable_pin_with_proper_timing_for_HD44780"""
    sleep(E_DELAY)
    GPIO.output(LCD_E, True)
    sleep(E_PULSE)
    GPIO.output(LCD_E, False)
    sleep(E_DELAY)

def lcd_send_byte(bits, mode):
    """Send_byte_to_LCD_in_4-bit_mode_with_proper_timing"""
    GPIO.output(LCD_RS, mode)
    sleep(E_DELAY)

    # High nibble
    GPIO.output(LCD_D4, bool(bits & 0x10))
    GPIO.output(LCD_D5, bool(bits & 0x20))
    GPIO.output(LCD_D6, bool(bits & 0x40))
    GPIO.output(LCD_D7, bool(bits & 0x80))
    lcd_toggle_enable()

    # Low nibble
    GPIO.output(LCD_D4, bool(bits & 0x01))
    GPIO.output(LCD_D5, bool(bits & 0x02))
    GPIO.output(LCD_D6, bool(bits & 0x04))
    GPIO.output(LCD_D7, bool(bits & 0x08))
    lcd_toggle_enable()

def lcd_init():
    """Robust_LCD_initialization_sequence_for_HD44780"""
    # Wait for LCD power-up (minimum 15ms after VCC rises to 4.5V)
    sleep(0.050) # 50ms to be safe

    # Initialize by instruction (4-bit interface)
    GPIO.output(LCD_RS, False)

    # First initialization: 8-bit mode

```

```

GPIO.output(LCD_D4, True)
GPIO.output(LCD_D5, True)
GPIO.output(LCD_D6, False)
GPIO.output(LCD_D7, False)
lcd_toggle_enable()
sleep(INIT_DELAY)

# Second initialization: 8-bit mode
lcd_toggle_enable()
sleep(INIT_DELAY)

# Third initialization: 8-bit mode
lcd_toggle_enable()
sleep(INIT_DELAY)

# Fourth initialization: Switch to 4-bit mode
GPIO.output(LCD_D4, False)
GPIO.output(LCD_D5, True)
GPIO.output(LCD_D6, False)
GPIO.output(LCD_D7, False)
lcd_toggle_enable()
sleep(INIT_DELAY)

# Function set: 4-bit, 2 lines, 5x8 font
lcd_send_byte(0x28, LCD_CMD)
sleep(CMD_DELAY)

# Display control: Display on, cursor off, blink off
lcd_send_byte(0x0C, LCD_CMD)
sleep(CMD_DELAY)

# Entry mode: Increment cursor, no shift
lcd_send_byte(0x06, LCD_CMD)
sleep(CMD_DELAY)

# Clear display
lcd_send_byte(0x01, LCD_CMD)
sleep(CMD_DELAY)

print("LCD_initialized_successfully")

def lcd_string(message, line):
    """Send_string_to_LCD_with_proper_timing"""
    message = message.ljust(LCD_WIDTH, " ")[:LCD_WIDTH]
    lcd_send_byte(line, LCD_CMD)
    sleep(CMD_DELAY)

    for ch in message:
        lcd_send_byte(ord(ch), LCD_CHR)
        sleep(CHAR_DELAY)

```

```

def lcd_clear():
    """Clear_LCD_display"""
    lcd_send_byte(0x01, LCD_CMD)
    sleep(CMD_DELAY)

# -----
# GPIO SETUP WITH PROPER INITIALIZATION
# -----
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

pins = [LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7,
        GREEN_LED, RED_LED, BUZZER]

for p in pins:
    GPIO.setup(p, GPIO.OUT)
    GPIO.output(p, False)

sleep(0.1)
lcd_init()

# -----
# GREEN LED CONTINUOUS BLINK THREAD
# -----
class GreenLEDController:
    def __init__(self):
        self.running = True
        self.enabled = True

    def worker(self):
        """Continuous_blink_thread_for_green_LED"""
        while self.running:
            if self.enabled:
                GPIO.output(GREEN_LED, True)
                sleep(GREEN_BLINK_RATE)
                GPIO.output(GREEN_LED, False)
                sleep(GREEN_BLINK_RATE)
            else:
                sleep(0.1)

    def start(self):
        self.thread = threading.Thread(target=self.worker, daemon=True)
        self.thread.start()

    def stop(self):
        self.running = False
        GPIO.output(GREEN_LED, False)

# -----

```

```
# VIRTUAL SERIAL HANDLER
```

```
#
```

```
class VirtualSerialReceiver:
    def __init__(self):
        self.current_state = SystemState.WAITING_MODEL
        self.last_message = ""
        self.running = False
        self.model_started = False
        self.message_count = 0
        self.last_read_time = time()
        self.init_files()

    def init_files(self):
        """Clear any existing data in serial files"""
        try:
            with open(VIRTUAL_SERIAL_RX, 'w') as f:
                f.write("")
            with open(VIRTUAL_SERIAL_TX, 'w') as f:
                f.write("LCD_READY\n")
            print("Virtual serial initialized")
        except Exception as e:
            print(f"File I/O error: {e}")

    def decode_message(self, msg):
        """Decode opcodes from Python model"""
        opcode_map = {
            "BOOT": "SYSTEM_BOOTING",
            "INIT": "INITIALIZING",
            "LOAD": "LOADING",
            "PROC": "PROCESSING",
            "TRAIN": "TRAINING",
            "RDY": "READY",
            "NORM": "NORMAL",
            "WARN": "PRE-SEIZURE",
            "SEIZ": "SEIZURE",
            "REST": "RESTARTING",
            "HB": "HEARTBEAT"
        }
        return opcode_map.get(msg.strip(), msg)

    def worker(self):
        """Background thread for reading serial data"""
        self.running = True
        last_content = ""
        no_data_counter = 0

        while self.running:
            try:
                current_time = time()
```

```

if os.path.exists(VIRTUAL_SERIAL_RX):
    with open(VIRTUAL_SERIAL_RX, 'r') as f:
        content = f.read().strip()

    if content and content != last_content:
        last_content = content
        no_data_counter = 0

    lines = content.split('\n')
    if lines:
        msg = self.decode_message(lines[-1])

        if msg == "HEARTBEAT":
            continue

        if not self.model_started and msg != "HEARTBEAT":
            self.model_started = True
            print("Model_connected")

        self.last_message = msg
        self.message_count += 1
        self.last_read_time = current_time
        print(f"[{self.message_count}]_{msg}")

        # Update state
        state_map = {
            "SYSTEM_BOOTING": SystemState.BOOT,
            "INITIALIZING": SystemState.INIT,
            "LOADING": SystemState.LOADING,
            "PROCESSING": SystemState.PROCESSING,
            "TRAINING": SystemState.TRAINING,
            "READY": SystemState.READY,
            "NORMAL": SystemState.NORMAL,
            "PRE-SEIZURE": SystemState.PRE_SEIZURE,
            "SEIZURE": SystemState.SEIZURE,
            "RESTARTING": SystemState.RESTARTING
        }

        if msg in state_map:
            self.current_state = state_map[msg]
        else:
            no_data_counter += 1
            if no_data_counter > 50 and not self.model_started:
                self.current_state = SystemState.WAITING_MODEL

    sleep(0.1)

except Exception as e:
    print(f"Read_error:_{e}")
    sleep(1)

```

```

def start(self):
    self.thread = threading.Thread(target=self.worker, daemon=True)
    self.thread.start()

```

```

def stop(self):
    self.running = False

```

```

# -----
# DISPLAY CONTROLLER
# -----

```

```

class DisplayController:

```

```

    def __init__(self, serial_rx):
        self.serial_rx = serial_rx
        self.last_update = 0
        self.last_state = None
        self.blink_state = False
        self.scroll_pos = 0
        self.scroll_text = "___SEEK_MEDICAL_HELP_NOW!___"
        self.loading_dots = 0
        self.animation_counter = 0

```

```

    def update(self):
        current_time = time()

        if current_time - self.last_update < LCD_REFRESH_RATE:
            return

```

```

        state = self.serial_rx.current_state

```

```

        if state in [SystemState.LOADING, SystemState.PROCESSING,
                     SystemState.TRAINING, SystemState.SEIZURE]:
            self.render_state(state)
        elif state != self.last_state:
            self.last_state = state
            self.render_state(state)

```

```

        self.last_update = current_time

```

```

    def render_state(self, state):
        """Render_LCD_and_control_LEDs/Buzzer"""

```

```

        if state == SystemState.WAITING_MODEL:
            lcd_clear()
            lcd_string("LCD_READY", LCD_LINE_1)
            lcd_string("Waiting_Model..", LCD_LINE_2)
            GPIO.output(RED_LED, False)
            GPIO.output(BUZZER, False)

```

```

        elif state == SystemState.BOOT:

```

```

        lcd_clear()
        lcd_string("SYSTEM_BOOTING", LCD_LINE_1)
        lcd_string("Please_Wait...", LCD_LINE_2)
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.INIT:
        lcd_clear()
        lcd_string("INITIALIZING", LCD_LINE_1)
        lcd_string("Setting_Up...", LCD_LINE_2)
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.LOADING:
        dots = "." * ((self.loading_dots % 4) + 1)
        lcd_string("LOADING" + dots.ljust(9), LCD_LINE_1)
        lcd_string("Reading_Data...", LCD_LINE_2)
        self.loading_dots += 1
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.PROCESSING:
        dots = "." * ((self.loading_dots % 4) + 1)
        lcd_string("PROCESSING" + dots.ljust(6), LCD_LINE_1)
        lcd_string("Analyzing...", LCD_LINE_2)
        self.loading_dots += 1
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.TRAINING:
        dots = "." * ((self.loading_dots % 4) + 1)
        lcd_string("TRAINING" + dots.ljust(8), LCD_LINE_1)
        lcd_string("Learning_Model", LCD_LINE_2)
        self.loading_dots += 1
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.READY:
        lcd_clear()
        lcd_string("SYSTEM_READY", LCD_LINE_1)
        lcd_string("Starting_Scan", LCD_LINE_2)
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

    elif state == SystemState.NORMAL:
        lcd_clear()
        lcd_string("STATUS:_NORMAL", LCD_LINE_1)
        lcd_string("All_Stable", LCD_LINE_2)
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

```

```

    elif state == SystemState.PRE_SEIZURE:
        lcd_clear()
        lcd_string("!PRE-SEIZURE!", LCD_LINE_1)
        lcd_string("Take_Meds_Now!", LCD_LINE_2)
        GPIO.output(RED_LED, self.blink_state)
        GPIO.output(BUZZER, self.blink_state)
        self.blink_state = not self.blink_state

    elif state == SystemState.SEIZURE:
        if self.blink_state:
            lcd_string("!!_SEIZURE_!!", LCD_LINE_1)
        else:
            lcd_string("", LCD_LINE_1)

        display = self.scroll_text[self.scroll_pos:self.scroll_pos + LCD_WIDTH]
        if len(display) < LCD_WIDTH:
            display += self.scroll_text[:LCD_WIDTH - len(display)]
        lcd_string(display, LCD_LINE_2)

        self.blink_state = not self.blink_state
        self.scroll_pos = (self.scroll_pos + 1) % len(self.scroll_text)

        GPIO.output(RED_LED, self.blink_state)
        GPIO.output(BUZZER, self.blink_state)

    elif state == SystemState.RESTARTING:
        lcd_clear()
        lcd_string("RESTARTING", LCD_LINE_1)
        lcd_string("Please_Wait...", LCD_LINE_2)
        GPIO.output(RED_LED, False)
        GPIO.output(BUZZER, False)

# -----
# MAIN LOOP
# -----
try:
    green_led = GreenLEDController()
    green_led.start()

    lcd_clear()
    lcd_string("System_Booting..", LCD_LINE_1)
    lcd_string("v2.0_Starting", LCD_LINE_2)
    sleep(2)

    serial_rx = VirtualSerialReceiver()
    display = DisplayController(serial_rx)

    lcd_clear()
    lcd_string("Initialized!", LCD_LINE_1)

```

```

sleep(1)

serial_rx.start()

lcd_clear()
lcd_string("LCD_READY", LCD_LINE_1)
lcd_string("Waiting_Model..", LCD_LINE_2)

print("LCD_Controller_Active")
print("Green_LED:_Continuous_blink")
print("Waiting_for_model...")

while True:
    display.update()
    sleep(0.05)

except KeyboardInterrupt:
    lcd_clear()
    lcd_string("System_Halted", LCD_LINE_1)
    green_led.stop()
    GPIO.output(RED_LED, False)
    GPIO.output(BUZZER, False)
    serial_rx.stop()
    GPIO.cleanup()
    print("Stopped")

```

.1.2 EEG Detection Model

Listing 2: Real-Time EEG Detection Model with Validation

```

# -----
# MERGED: Real-Time EEG Detection + Full Validation + LCD Output
# Combines visualization, validation metrics, and Proteus LCD states
# -----

import os, time, sys, warnings, threading, queue
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt, welch
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                                f1_score, confusion_matrix)

import seaborn as sns

```

```

warnings.filterwarnings("ignore")

# ===== CONFIGURATION =====
fs = 128
fLow, fHigh, nOrder = 0.5, 45, 4
nperseg = 128
parentPath = r"C:\Users\thebe\Documents\MATLAB\Bonn_Univeristy_Dataset"
update_rate = 0.15
watchdog_timeout = 2.0
MIN_SAMPLES_PER_CLASS = 5

VIRTUAL_SERIAL_RX = r"C:\Users\thebe\AppData\Local\Temp\vsm_serial_rx.txt"
VIRTUAL_SERIAL_TX = r"C:\Users\thebe\AppData\Local\Temp\vsm_serial_tx.txt"

CLASS_MAP = {"O": 0, "N": 0, "S": 1, "F": 1, "Z": 2}
CLASSES = ["Normal", "Pre-Seizure", "Seizure"]
COLORS = ["#00CC66", "#FFD633", "#FF3333"]
# =====

# ===== VIRTUAL SERIAL HANDLER =====
class VirtualSerialHandler:
    def __init__(self):
        self.tx_queue = queue.Queue(maxsize=10)
        self.last_heartbeat = time.time()
        self.running = False
        self.init_files()

    def init_files(self):
        try:
            with open(VIRTUAL_SERIAL_RX, 'w') as f:
                f.write("")
            with open(VIRTUAL_SERIAL_TX, 'w') as f:
                f.write("")
            print("Virtual_serial_initialized")
            self.handshake()
        except Exception as e:
            print(f"File_init_error:_{e}")

    def handshake(self):
        self.send_raw("READY")
        timeout = time.time() + 5
        while time.time() < timeout:
            try:
                with open(VIRTUAL_SERIAL_TX, 'r') as f:
                    response = f.read().strip()
                if "ACK" in response:
                    print("Handshake_established")
                    self.last_heartbeat = time.time()
                    return True
            except:

```

```

        pass
        time.sleep(0.2)
    print("Handshake_timeout, _continuing_anyway...")
    return False

def send_raw(self, msg):
    try:
        opcode_map = {
            "SYSTEM_BOOTING": "BOOT", "INITIALIZING": "INIT",
            "LOADING": "LOAD", "PROCESSING": "PROC",
            "TRAINING": "TRAIN", "READY": "RDY",
            "NORMAL": "NORM", "PRE-SEIZURE": "WARN",
            "SEIZURE": "SEIZ", "RESTARTING": "REST",
            "HEARTBEAT": "HB"
        }
        compressed = opcode_map.get(msg, msg[:16])

        with open(VIRTUAL_SERIAL_RX, 'w') as f:
            f.write(compressed + "\n")

        self.last_heartbeat = time.time()
        print(f"{msg}_{compressed}")
    except Exception as e:
        print(f"Send_error:{e}")

def send(self, msg):
    if not self.tx_queue.full():
        self.tx_queue.put(msg)
    else:
        self.send_raw(msg)

def worker(self):
    self.running = True
    while self.running:
        try:
            msg = self.tx_queue.get(timeout=0.5)
            self.send_raw(msg)
            time.sleep(update_rate)
        except queue.Empty:
            if time.time() - self.last_heartbeat > 1.0:
                self.send_raw("HEARTBEAT")

def start(self):
    self.thread = threading.Thread(target=self.worker, daemon=True)
    self.thread.start()

def stop(self):
    self.running = False

serial_handler = VirtualSerialHandler()

```

```

serial_handler.start()

# ===== SIGNAL PROCESSING =====
def bandpass_filter(sig):
    b, a = butter(nOrder, [fLow/(0.5*fs), fHigh/(0.5*fs)], btype="band")
    return filtfilt(b, a, sig)

def extract_feature(fpath):
    try:
        data = np.loadtxt(fpath)
        if len(data) > 4096:
            data = data[:, :2]

        filt = bandpass_filter(data)
        f, psd = welch(filt, fs=fs, nperseg=nperseg)

        delta = np.mean(psd[(f >= 1) & (f <= 4)])
        theta = np.mean(psd[(f >= 4) & (f <= 8)])
        alpha = np.mean(psd[(f >= 8) & (f <= 13)])
        beta = np.mean(psd[(f >= 13) & (f <= 30)])

        return [delta, theta, alpha, beta]
    except Exception as e:
        return None

def hybrid_predict(x_vec, scaler, svmModel, rfModel):
    xs = scaler.transform([x_vec])
    svm_conf = svmModel.predict_proba(xs)[0]
    rf_conf = rfModel.predict_proba([x_vec])[0]
    combined_conf = 0.4 * svm_conf + 0.6 * rf_conf
    final_pred = int(np.argmax(combined_conf))
    return final_pred, combined_conf

# ===== BASELINE COMPARISON =====
def train_baselines(X_train, X_test, y_train, y_test, scaler):
    print("\n" + "="*60)
    print("BASELINE_MODEL_COMPARISON")
    print("="*60)

    results = {}
    X_train_scaled = scaler.transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Majority Class
    y_majority = np.full(len(y_test), np.bincount(y_train).argmax())
    results['Majority_Class'] = {
        'predictions': y_majority,
        'accuracy': accuracy_score(y_test, y_majority),
        'train_accuracy': accuracy_score(y_train,
            np.full(len(y_train), np.bincount(y_train).argmax()))
    }

```

```

}

# Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
results['Logistic_Regression'] = {
    'model': lr,
    'predictions': lr.predict(X_test_scaled),
    'accuracy': accuracy_score(y_test, lr.predict(X_test_scaled)),
    'train_accuracy': accuracy_score(y_train, lr.predict(X_train_scaled))
}

# SVM
svm = SVC(kernel="rbf", C=2, gamma="scale", probability=True,
           class_weight="balanced", random_state=42)
svm.fit(X_train_scaled, y_train)
results['SVM'] = {
    'model': svm,
    'predictions': svm.predict(X_test_scaled),
    'accuracy': accuracy_score(y_test, svm.predict(X_test_scaled)),
    'train_accuracy': accuracy_score(y_train, svm.predict(X_train_scaled))
}

# Random Forest
rf = RandomForestClassifier(n_estimators=100, max_depth=20,
                           class_weight="balanced", random_state=42)
rf.fit(X_train, y_train)
results['Random_Forest'] = {
    'model': rf,
    'predictions': rf.predict(X_test),
    'accuracy': accuracy_score(y_test, rf.predict(X_test)),
    'train_accuracy': accuracy_score(y_train, rf.predict(X_train))
}

# Hybrid
svm_proba = svm.predict_proba(X_test_scaled)
rf_proba = rf.predict_proba(X_test)
hybrid_proba = 0.4 * svm_proba + 0.6 * rf_proba
y_pred_hybrid = np.argmax(hybrid_proba, axis=1)

results['Hybrid_(SVM+RF)'] = {
    'predictions': y_pred_hybrid,
    'accuracy': accuracy_score(y_test, y_pred_hybrid),
    'train_accuracy': accuracy_score(y_train,
                                     np.argmax(0.4 * svm.predict_proba(X_train_scaled) +
                                               0.6 * rf.predict_proba(X_train), axis=1))
}

# Print comparison
for name, res in results.items():

```

```

    train_acc = res['train_accuracy']
    test_acc = res['accuracy']
    gap = train_acc - test_acc
    print(f"{name:20s}_Train:_{train_acc*100:5.1f}%_|_"
          f"Test:_{test_acc*100:5.1f}%_|_Gap:_{gap*100:4.1f}%")

    return results

# ===== VISUALIZATION SETUP =====
plt.ion()
fig = plt.figure(figsize=(16, 9))
gs = fig.add_gridspec(3, 3, hspace=0.35, wspace=0.35)

ax1 = fig.add_subplot(gs[0, 0])
bars = ax1.bar(CLASSES, [0, 0, 0], color=COLORS)
ax1.set_ylim(0, 1)
ax1.set_ylabel("Confidence")
ax1.set_title("Current_Prediction")

ax2 = fig.add_subplot(gs[1:, :])
timeline_x, timeline_y = [], [], []
lines = [ax2.plot([], [], color=c, label=lbl, linewidth=2)[0]
          for c, lbl in zip(COLORS, CLASSES)]
ax2.set_xlim(0, 50)
ax2.set_ylim(0, 1)
ax2.set_xlabel("Sample_Index")
ax2.set_ylabel("Confidence_Over_Time")
ax2.legend()

ax3 = fig.add_subplot(gs[0, 1])
ax3.set_title("Confusion_Matrix")
ax3.axis('off')

ax4 = fig.add_subplot(gs[0, 2])
ax4.set_title("Performance_Metrics")
ax4.axis('off')

fig.canvas.draw_idle()
plt.show(block=False)
plt.pause(0.1)

def update_plot(conf, index):
    for bar, val in zip(bars, conf):
        bar.set_height(val)

    timeline_x.append(index)
    for i in range(3):
        timeline_y[i].append(conf[i])
        lines[i].set_data(timeline_x, timeline_y[i])

```

```

    if index > ax2.get_xlim()[1]:
        ax2.set_xlim(0, index + 10)

    fig.canvas.draw_idle()
    fig.canvas.flush_events()

def plot_confusion_matrix(y_true, y_pred):
    ax3.clear()
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=CLASSES, yticklabels=CLASSES,
                cbar=True, ax=ax3, square=True)
    ax3.set_xlabel('Predicted')
    ax3.set_ylabel('True')
    ax3.set_title('Confusion_Matrix')
    fig.canvas.draw_idle()

def plot_metrics(acc, prec, rec, f1, specificity):
    ax4.clear()
    metrics = ['Accuracy', 'Precision', 'Recall', 'F1', 'Specificity']
    values = [acc, prec, rec, f1, specificity]
    colors_m = ['#00CC66' if v >= 0.9 else '#FFD633' if v >= 0.7
                else '#FF3333' for v in values]
    bars_m = ax4.barh(metrics, values, color=colors_m)
    ax4.set_xlim(0, 1)
    ax4.set_title('Performance_Metrics')
    for i, (bar, val) in enumerate(zip(bars_m, values)):
        ax4.text(val + 0.02, i, f'{val*100:.1f}%', va='center', fontsize=8)
    fig.canvas.draw_idle()

# ===== MAIN LOOP =====
try:
    while True:
        print("\n" + "="*70)
        print("SYSTEM_BOOTING...")
        print("="*70)
        serial_handler.send_raw("SYSTEM_BOOTING")
        time.sleep(2)

        serial_handler.send_raw("INITIALIZING")
        time.sleep(1)

        print("\nLOADING_DATA...")
        serial_handler.send_raw("LOADING")

    X, y, file_paths = [], [], []

    for subset, label in CLASS_MAP.items():
        path = os.path.join(parentPath, subset)
        if not os.path.exists(path):

```

```

        continue

    files = [f for f in os.listdir(path) if f.endswith(".txt")]
    print(f"__Class_{subset}__>_{len(files)}_files")

    for fname in files:
        file_paths.append((os.path.join(path, fname), label))

    print(f"Found_{len(file_paths)}_samples")
    time.sleep(1)

    print("\nPROCESSING_DATA...")
    serial_handler.send_raw("PROCESSING")

    for i, (fpath, label) in enumerate(file_paths):
        feats = extract_feature(fpath)
        if feats is not None:
            X.append(feats)
            y.append(label)

        if (i+1) % 50 == 0:
            print(f"_{i+1}/{len(file_paths)}_processed...")

    print(f"_{len(X)}_samples_ready")
    time.sleep(1)

    X_arr, y_arr = np.array(X), np.array(y)

    if len(X_arr) < 50:
        print("Insufficient_data")
        time.sleep(5)
        continue

    print("\nSPLITTING_DATA...")
    unique_classes = np.unique(y_arr)
    n_splits = min(3, min([np.sum(y_arr == c) for c in unique_classes]))

    if n_splits >= 2:
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
                               random_state=42)
        train_idx, test_idx = next(skf.split(X_arr, y_arr))
    else:
        split = int(0.8 * len(X_arr))
        train_idx = list(range(split))
        test_idx = list(range(split, len(X_arr)))

    X_train, X_test = X_arr[train_idx], X_arr[test_idx]
    y_train, y_test = y_arr[train_idx], y_arr[test_idx]

    print(f"__Train:_{len(X_train)}_|_Test:_{len(X_test)}")

```

```

scaler = StandardScaler()
scaler.fit(X_train)

print("\nTRAINING_MODELS...")
serial_handler.send_raw("TRAINING")

results = train_baselines(X_train, X_test, y_train, y_test, scaler)

svmModel = results['SVM']['model']
rfModel = results['Random_Forest']['model']

print("\nTraining_complete!")
time.sleep(1)

print("\n" + "="*70)
print("STARTING_REAL-TIME_DETECTION_WITH_LIVE_VISUALIZATION")
print("="*70)
serial_handler.send_raw("READY")
time.sleep(1)

timeline_x.clear()
for t in timeline_y:
    t.clear()

predictions = []
prev_conf = np.zeros(3)

for index, (feats, true_label) in enumerate(zip(X, y)):
    pred, conf = hybrid_predict(feats, scaler, svmModel, rfModel)

    conf = 0.7 * prev_conf + 0.3 * conf
    prev_conf = conf

    predictions.append(pred)

    if pred == 0:
        serial_handler.send_raw("NORMAL")
        state_str = "NORMAL"
    elif pred == 1:
        serial_handler.send_raw("PRE-SEIZURE")
        state_str = "PRE-SEIZURE"
    else:
        serial_handler.send_raw("SEIZURE")
        state_str = "SEIZURE"

    update_plot(conf, index)

    if index % 10 == 0:
        print(f"Sample_{index:3d}:_{state_str:12s}_|_|")

```

```

        f"Conf:_{conf[pred]*100:5.1f}%")

    time.sleep(update_rate)

    print("\n" + "="*70)
    print("CALCULATING_FINAL_METRICS...")
    print("="*70)

    y_pred = rfModel.predict(X_arr)

    cm = confusion_matrix(y_arr, y_pred)
    plot_confusion_matrix(y_arr, y_pred)

    acc = accuracy_score(y_arr, y_pred)
    prec = precision_score(y_arr, y_pred, average='weighted',
                           zero_division=0)
    rec = recall_score(y_arr, y_pred, average='weighted',
                       zero_division=0)
    f1 = f1_score(y_arr, y_pred, average='weighted', zero_division=0)

    seizure_recall = recall_score(y_arr, y_pred, labels=[2],
                                  average='macro',
                                  zero_division=0) if 2 in y_arr else 0

    if 0 in y_arr and cm.shape[0] > 0:
        tn = cm[0, 0]
        fp = cm[0, 1:].sum() if cm.shape[1] > 1 else 0
        specificity = tn / (tn + fp) if (tn + fp) > 0 else 0
    else:
        specificity = 0

    print(f"\nOverall_Metrics:")
    print(f"___Accuracy:___{acc*100:.2f}%")
    print(f"___Precision:___{prec*100:.2f}%")
    print(f"___Recall:___{rec*100:.2f}%")
    print(f"___F1_Score:___{f1*100:.2f}%")
    print(f"___Seizure_Recall:___{seizure_recall*100:.2f}%")
    print(f"___Specificity:___{specificity*100:.2f}%")

    plot_metrics(acc, prec, rec, f1, specificity)

    print("\n" + "="*70)
    print("VALIDATION_SUMMARY")
    print("="*70)

    hybrid_gap = (results['Hybrid_(SVM+RF)']['train_accuracy'] -
                  results['Hybrid_(SVM+RF)']['accuracy'])
    print(f"\nHybrid_Model_Generalization:")
    print(f"___Train_Accuracy:_"
          f"{results['Hybrid_(SVM+RF)']['train_accuracy']*100:.2f}%")

```

```

print (f"Test_Accuracy: "
        f"{results['Hybrid_(SVM+RF)']['accuracy']*100:.2f}%")
print (f"Gap: {hybrid_gap*100:.2f}%")

if hybrid_gap < 0.03:
    print (f"Status: _GOOD_ _Excellent_generalization")
elif hybrid_gap < 0.05:
    print (f"Status: _ACCEPTABLE_ _Moderate_generalization")
else:
    print (f"Status: _WARNING_ _Potential_overfitting")

print ("\n" + "="*70)
print ("RESTARTING_IN_10_SECONDS... _ (Press_Ctrl+C_to_stop)")
print ("="*70)
serial_handler.send_raw("RESTARTING")
time.sleep(10)

except KeyboardInterrupt:
    serial_handler.send_raw("SYSTEM_STOPPED")
    serial_handler.stop()
    plt.close()
    print ("\nSystem_stopped_by_user")
    sys.exit(0)
except Exception as e:
    print (f"\nCritical_error:_{e}")
    import traceback
    traceback.print_exc()
    serial_handler.stop()
    plt.close()
    sys.exit(1)

```

Assessed By

Supervisor (Name & Signature)



Prof. Abid Yahya

Co Supervisor (Name & Signature)



Dr Mtengi